



Instituto Politécnico Nacional



Unidad Profesional Interdisciplinaria de
Ingeniería y Ciencias Sociales y
Administrativas

Estructura de datos

Secuencia: 2NM23

Profesor: Yventz Entzana Garduño

Prácticas 2do parcial.

Integrantes:	No.lista:
--------------	-----------

Morales Trejo Dante	25
---------------------	----

Roldan Medrano Pablo	35
----------------------	----

Torres López Javier	39
---------------------	----

Repositorio GitHub:

<https://github.com/dmt947/practicas-estructura-datos-2nm23>

Tabla de contenido

Práctica 17 – Ordenamiento (Merge Sort)	6
Práctica 18 – Ordenamiento (Quick Sort).....	9
Práctica 19 – Ordenamiento Indirecto (Burbuja)	12
Práctica 20 – Ordenamiento Indirecto (Merge Sort).....	15
Práctica 21 – Ordenamiento Indirecto (Quick Sort)	18
Práctica 1 – Punteros con Clases/Estructuras	21
Práctica 3 – Pila Estática.....	23
Práctica 4 – Cola Estática	27
Práctica 5 – Lista Estática	31
Práctica 6 – Pila Dinámica (Punteros)	35
Práctica 7 – Cola Dinámica (Punteros)	39
Práctica 7 bis – Lista Dinámica	43
Práctica 8 – Pila con Librerías/Clases.....	47
Práctica 9 – Cola con Librerías/Clases	49
Práctica 10 – Lista con Librerías/Clases	51
Práctica 10 – Promedio con puntero al arreglo (parcial 1).....	53
Práctica 13 – Nuevo tipo de dato con puntero al arreglo (parcial 1).....	55

Práctica 16 – Ordenamiento (Burbuja)

Funcionamiento

El algoritmo de burbuja compara pares de elementos adyacentes y los intercambia si están en el orden incorrecto. Repite este proceso hasta que el arreglo queda ordenado. Se puede aplicar a caracteres, enteros o tipos de datos definidos (estructuras/clases).

Errores

- No controlar correctamente los límites del arreglo.
- Comparaciones incorrectas en tipos de datos personalizados.
- Ineficiencia con grandes volúmenes de datos.

```
1  #include <iostream>
2  #include <vector>
3
4  #include "Burbuja.h"
5  #include "Utilidades.h"
6  #include "persona.h"
7
8  using namespace std;
9
10 int main(){
11     int CA,TV;
12
13     cout<<"Cantidad de valores\n";
14     cin>>CA;
15
16     cout<<"Tipo Entero(1)/Float(2)/String(3)/Persona(4)\n";
17     cin>>TV;
18
19     if(TV == 1){
20         vector<int> v(CA);
21         Utilidades<int>::leer(v);
22
23         IOordenador<int>* ord = new Burbuja<int>(); // POLIMORFISMO
24
25         Utilidades<int>::imprimir(v);
26         ord->ordenar(v);
27         Utilidades<int>::imprimir(v);
28
29         delete ord;
30     }
31     else if(TV == 2){
```

```
32         vector<float> v(CA);
33         Utilidades<float>::leer(v);
34
35         IOordenador<float>* ord = new Burbuja<float>();
36
37         Utilidades<float>::imprimir(v);
38         ord->ordenar(v);
39         Utilidades<float>::imprimir(v);
40
41         delete ord;
42     }
43     else if(TV == 3){
44         vector<string> v(CA);
45         Utilidades<string>::leerString(v);
46
47         IOordenador<string>* ord = new Burbuja<string>();
48
49         Utilidades<string>::imprimir(v);
50         ord->ordenar(v);
51         Utilidades<string>::imprimir(v);
52
53         delete ord;
54     }
55     else if(TV == 4){
56         vector<persona> v(CA);
57
58         cout<<"Ordenar por Nombre(1)/Edad(2)\n";
59         cin>>persona::cr;
60     }
```

```
61         Utilidades<persona>::leer(v);
62
63         IOordenador<persona>* ord = new Burbuja<persona>();
64
65         Utilidades<persona>::imprimir(v);
66         ord->ordenar(v);
67         Utilidades<persona>::imprimir(v);
68
69         delete ord;
70     }
71
72     return 0;
73 }
```

```

1  #ifndef BURBUJA_H
2  #define BURBUJA_H
3
4  #include "Ordenador.h"
5  #include <vector>
6  using namespace std;
7
8  template<typename T>
9  class Burbuja : public Ordenador<T>{
10 public:
11     void ordenar(vector<T>& datos) override;
12 };
13
14 #include "Burbuja.tpp"
15
16 #endif

```

```

1  #ifndef BURBUJA_TPP
2  #define BURBUJA_TPP
3
4  #include <vector>
5  using namespace std;
6
7  template<typename T>
8  void Burbuja<T>::ordenar(vector<T>& datos){
9      int n = datos.size();
10
11      for(int i = 0; i < n; i++){
12          for(int j = 0; j < n - 1 - i; j++){
13              if(datos[j] > datos[j + 1]){
14                  this->intercambio(datos[j], datos[j + 1]);
15              }
16          }
17      }
18  }
19
20 #endif

```

```

1  #ifndef IORDENADOR_H
2  #define IORDENADOR_H
3
4  #include <vector>
5  using namespace std;
6
7  template<typename T>
8  class IOrdenador{
9  public:
10     virtual void ordenar(vector<T>& datos) = 0;
11     virtual ~IOrdenador() {}
12 };
13
14 #endif

```

```

1  #ifndef ORDENADOR_H
2  #define ORDENADOR_H
3
4  #include "IOrdenador.h"
5
6  template<typename T>
7  class Ordenador : public IOrdenador<T>{
8  protected:
9      void intercambio(T &a, T &b);
10 };
11
12 #include "Ordenador.tpp"
13
14 #endif

```

```

1  #ifndef UTILIDADES_H
2  #define UTILIDADES_H
3
4  #include <vector>
5  #include <string>
6  using namespace std;
7
8  template<typename T>
9  class Utilidades{
10 public:
11     static void leer(vector<T>& v);
12     static void leerString(vector<T>& v);
13     static void imprimir(const vector<T>& v);
14 };
15
16 #include "Utilidades.tpp"
17
18 #endif

```

```

1  #ifndef UTILIDADES_TPP
2  #define UTILIDADES_TPP
3
4  #include <iostream>
5  #include <vector>
6  #include <string>
7  using namespace std;
8
9  template<typename T>
10 void Utilidades<T>::leer(vector<T>& v){
11     cout << "Introduzca valores\n";
12     for(int i = 0; i < v.size(); i++){
13         cin >> v[i];
14     }
15 }
16
17 template<typename T>
18 void Utilidades<T>::leerString(vector<T>& v){
19     cout << "Introduzca valores\n";
20     for(size_t i = 0; i < v.size(); i++){
21         getline(cin >> ws, v[i]);
22     }
23 }
24
25 template<typename T>
26 void Utilidades<T>::imprimir(const vector<T>& v){
27     for(int i = 0; i < v.size(); i++){
28         cout << v[i] << " ";
29     }
30     cout << endl;
31 }
32

```

```

1  #ifndef ORDENADOR_TPP
2  #define ORDENADOR_TPP
3
4  template<typename T>
5  void Ordenador<T>::intercambio(T &a, T &b){
6      T aux = a;
7      a = b;
8      b = aux;
9  }
10
11 #endif

```

```

1  #include "persona.h"
2
3  int persona::cr = 1;
4
5  persona::persona(){
6      nombre="";
7      edad=0;
8  }
9
10 string persona::getNombre() const{return nombre;}
11 int persona::getEdad() const{return edad;}
12
13 void persona::setNombre(const string& n){nombre=n;}
14 void persona::setEdad(int e){ if(e>=0) edad=e; }
15
16 ostream& operator<<(ostream &os, const persona &p){
17     os<<"Nombre: "<<p.getNombre()<<" Edad: "<<p.getEdad();
18     return os;
19 }
20
21 istream& operator>>(istream &is, persona &p){
22     string n;
23     int e;
24
25     cout<<"Nombre: ";
26     if(is.peek()=='\n') is.ignore();
27     getline(is,n);
28
29     cout<<"Edad: ";
30     is>>e;
31
32     p.setNombre(n);
33     p.setEdad(e);
34     return is;
35 }
36
37 bool persona::operator>(const persona& o){
38     if(cr==1) return nombre > o.nombre;
39     return edad > o.edad;
40 }

```

```

1  #ifndef PERSONA_H
2  #define PERSONA_H
3
4  #include <iostream>
5  #include <string>
6  using namespace std;
7
8  class persona{
9  private:
10     string nombre;
11     int edad;
12
13 public:
14     static int cr;
15
16     persona();
17
18     string getNombre() const;
19     int getEdad() const;
20
21     void setNombre(const string&);
22     void setEdad(int);
23
24     bool operator>(const persona&);
25
26     friend ostream& operator<<(ostream&, const persona&);
27     friend istream& operator>>(istream&, persona&);
28 };
29
30 #endif

```

Práctica 17 – Ordenamiento (Merge Sort)

Funcionamiento

Divide el arreglo en mitades hasta tener subarreglos de un elemento, luego los combina ordenadamente. Es un algoritmo eficiente basado en “divide y vencerás”.

Errores

- Mal manejo de índices al dividir y fusionar.
- Problemas con memoria.
- Errores al comparar clases.

```
1  #include <iostream>
2  #include <vector>
3  #include "Merge.h"
4  #include "Utilidades.h"
5  #include "persona.h"
6  using namespace std;
7
8  int main(){
9      int CA,TV;
10
11      cout<<"Cantidad de valores\n";
12      cin>>CA;
13
14      cout<<"Tipo Entero(1)/Float(2)/String(3)/Persona(4)\n";
15      cin>>TV;
16
17      if(TV == 1){
18          vector<int> v(CA);
19          Utilidades<int>::leer(v);
20
21          IOrdenador<int>* ord = new Merge<int>(); // POLIMORFISMO
22
23          Utilidades<int>::imprimir(v);
24          ord->ordenar(v);
25          Utilidades<int>::imprimir(v);
26
27          delete ord;
28      }
29      else if(TV == 2){
30          vector<float> v(CA);
31          Utilidades<float>::leer(v);
32
33          IOrdenador<float>* ord = new Merge<float>();
34
35          Utilidades<float>::imprimir(v);
36          ord->ordenar(v);
37          Utilidades<float>::imprimir(v);
38
39          delete ord;
40      }
41      else if(TV == 3){
42          vector<string> v(CA);
43          Utilidades<string>::leerString(v);
44
45          IOrdenador<string>* ord = new Merge<string>();
46
47          Utilidades<string>::imprimir(v);
48          ord->ordenar(v);
49          Utilidades<string>::imprimir(v);
50
51          delete ord;
52      }
53      else if(TV == 4){
54          vector<persona> v(CA);
55
56          cout<<"Ordenar por Nombre(1)/Edad(2)\n";
57          cin>>persona::cr;
58
59          Utilidades<persona>::leer(v);
60
61          IOrdenador<persona>* ord = new Merge<persona>();
```

```
62
63          Utilidades<persona>::imprimir(v);
64          ord->ordenar(v);
65          Utilidades<persona>::imprimir(v);
66
67          delete ord;
68      }
69
70      return 0;
71  }
```

```

1  #ifndef IORDENADOR_H
2  #define IORDENADOR_H
3
4  #include <vector>
5  using namespace std;
6
7  template<typename T>
8  class IOordenador{
9  public:
10     virtual void ordenar(vector<T>& datos) = 0;
11     virtual ~IOordenador() {}
12 };
13
14 #endif

```

```

1  #ifndef MERGE_H
2  #define MERGE_H
3
4  #include <vector>
5  using namespace std;
6
7  // Clase Merge
8  template<typename T>
9  class Merge : public Ordenador<T> {
10 public:
11     void ordenar(vector<T>& v) override;
12
13 private:
14     void merge(vector<T>& n, int inicio, int mitad, int final);
15     void mergeSort(vector<T>& n, int inicio, int final);
16 };
17
18 #include "Merge.tpp"
19
20 #endif

```

```

1  #ifndef MERGE_TPP
2  #define MERGE_TPP
3
4  #include "Merge.h"
5  template<typename T>
6  void Merge<T>::ordenar(vector<T>& v) {
7      if (v.empty()) return;
8      mergeSort(v, 0, v.size() - 1);
9  }
10
11  template<typename T>
12  void Merge<T>::mergeSort(vector<T>& n, int inicio, int final) {
13      if (inicio >= final) return;
14
15      int mitad = inicio + (final - inicio) / 2;
16
17      mergeSort(n, inicio, mitad);
18      mergeSort(n, mitad + 1, final);
19
20      merge(n, inicio, mitad, final);
21  }
22
23  template<typename T>
24  void Merge<T>::merge(vector<T>& n, int inicio, int mitad, int final) {
25      int i = inicio, j = mitad + 1, k = 0;
26
27      vector<T> aux(final - inicio + 1);
28
29      while (i <= mitad && j <= final) {
30          if (n[i] < n[j]) {
31              aux[k] = n[i];
32              i++;
33          } else {
34              aux[k] = n[j];
35              j++;
36          }
37          k++;
38      }
39
40      while (i <= mitad) {
41          aux[k] = n[i];
42          i++;
43          k++;
44      }
45
46      while (j <= final) {
47          aux[k] = n[j];
48          j++;
49          k++;
50      }
51
52      for (int h = 0; h < k; h++) {
53          n[inicio + h] = aux[h];
54      }
55  }
56
57 #endif

```

```

1  #ifndef ORDENADOR_H
2  #define ORDENADOR_H
3
4  #include "IOordenador.h"
5
6  template<typename T>
7  class Ordenador : public IOordenador<T>{
8  protected:
9      void intercambio(T &a, T &b);
10 };
11
12 #include "ordenador.tpp"
13
14 #endif

```

```

1  #ifndef ORDENADOR_TPP
2  #define ORDENADOR_TPP
3
4  template<typename T>
5  void Ordenador<T>::intercambio(T &a, T &b){
6      T aux = a;
7      a = b;
8      b = aux;
9  }
10
11 #endif

```

```

1  #ifndef UTILIDADES_H
2  #define UTILIDADES_H
3
4  #include <vector>
5  #include <string>
6  using namespace std;
7
8  template<typename T>
9  class Utilidades{
10 public:
11     static void leer(vector<T>& v);
12     static void leerString(vector<T>& v);
13     static void imprimir(const vector<T>& v);
14 };
15
16 #include "Utilidades.tpp"
17
18 #endif

```

```

1  #ifndef PERSONA_H
2  #define PERSONA_H
3
4  #include <iostream>
5  #include <string>
6  using namespace std;
7
8  class persona{
9  private:
10     string nombre;
11     int edad;
12
13 public:
14     static int cr;
15
16     persona();
17
18     string getNombre() const;
19     int getEdad() const;
20
21     void setNombre(const string&);
22     void setEdad(int);
23
24     bool operator>(const persona&);
25     bool operator<(const persona&);
26
27     friend ostream& operator<<(ostream&, const persona&);
28     friend istream& operator>>(istream&, persona&);
29 };
30
31 #endif

```

```

1  #ifndef UTILIDADES_TPP
2  #define UTILIDADES_TPP
3
4  #include <iostream>
5  #include <vector>
6  #include <string>
7  using namespace std;
8
9  template<typename T>
10 void Utilidades<T>::leer(vector<T>& v){
11     cout << "Introduzca valores\n";
12     for(int i = 0; i < v.size(); i++){
13         cin >> v[i];
14     }
15 }
16
17 template<typename T>
18 void Utilidades<T>::leerString(vector<T>& v){
19     cout << "Introduzca valores\n";
20     for(size_t i = 0; i < v.size(); i++){
21         getline(cin >> ws, v[i]);
22     }
23 }
24
25 template<typename T>
26 void Utilidades<T>::imprimir(const vector<T>& v){
27     for(int i = 0; i < v.size(); i++){
28         cout << v[i] << " ";
29     }
30     cout << endl;
31 }

```

```

1  #include "persona.h"
2
3  int persona::cr = 1;
4
5  persona::persona(){
6     nombre="";
7     edad=0;
8 }
9
10 string persona::getNombre() const{return nombre;}
11 int persona::getEdad() const{return edad;}
12
13 void persona::setNombre(const string& n){nombre=n;}
14 void persona::setEdad(int e){ if(e=0) edad=e; }
15
16 ostream& operator<<(ostream& os, const persona& p){
17     os<<"Nombre: "<<p.getNombre()<<" Edad: "<<p.getEdad();
18     return os;
19 }
20
21 istream& operator>>(istream& is, persona& p){
22     string n;
23     int e;
24
25     cout<<"Nombre: ";
26     if(is.peek()=='\n') is.ignore();
27     getline(is,n);
28
29     cout<<"Edad: ";
30     is>>e;
31
32     p.setNombre(n);
33     p.setEdad(e);
34     return is;
35 }
36
37 bool persona::operator>(const persona& o){
38     if(cr==1) return nombre > o.nombre;
39     return edad > o.edad;
40 }
41
42 bool persona::operator<(const persona& o){
43     if(cr==1) return nombre < o.nombre;
44     return edad < o.edad;
45 }

```


Práctica 18 – Ordenamiento (Quick Sort)

Funcionamiento

Selecciona un pivote y reorganiza el arreglo colocando menores a la izquierda y mayores a la derecha. Luego aplica recursividad.

Errores

- Desbordamiento de pila por recursividad.
- Errores en partición del arreglo.

```
1  #include <iostream>
2  #include <vector>
3  #include "Quick.h"
4  #include "Utilidades.h"
5  #include "persona.h"
6  using namespace std;
7
8  int main(){
9      int CA,TV;
10
11      cout<<"Cantidad de valores\n";
12      cin>>CA;
13
14      cout<<"Tipo Entero(1)/Float(2)/String(3)/Persona(4)\n";
15      cin>>TV;
16
17      if(TV == 1){
18          vector<int> v(CA);
19          Utilidades<int>::leer(v);
20
21          IOrdenador<int>* ord = new Quick<int>(); // POLIMORFISMO
22
23          Utilidades<int>::imprimir(v);
24          ord->ordenar(v);
25          Utilidades<int>::imprimir(v);
26
27          delete ord;
28      }
29      else if(TV == 2){
30          vector<float> v(CA);
31          Utilidades<float>::leer(v);
32
```

```
33          IOrdenador<float>* ord = new Quick<float>();
34
35          Utilidades<float>::imprimir(v);
36          ord->ordenar(v);
37          Utilidades<float>::imprimir(v);
38
39          delete ord;
40      }
41      else if(TV == 3){
42          vector<string> v(CA);
43          Utilidades<string>::leerString(v);
44
45          IOrdenador<string>* ord = new Quick<string>();
46
47          Utilidades<string>::imprimir(v);
48          ord->ordenar(v);
49          Utilidades<string>::imprimir(v);
50
51          delete ord;
52      }
53      else if(TV == 4){
54          vector<persona> v(CA);
55
56          cout<<"Ordenar por Nombre(1)/Edad(2)\n";
57          cin>>persona::cr;
58
59          Utilidades<persona>::leer(v);
60
61          IOrdenador<persona>* ord = new Quick<persona>();
```

```
62
63          Utilidades<persona>::imprimir(v);
64          ord->ordenar(v);
65          Utilidades<persona>::imprimir(v);
66
67          delete ord;
68      }
69
70      return 0;
71  }
```

```

1  #ifndef IORDENADOR_H
2  #define IORDENADOR_H
3
4  #include <vector>
5  using namespace std;
6
7  template<typename T>
8  class IOrdenador{
9  public:
10     virtual void ordenar(vector<T>& datos) = 0;
11     virtual ~IOrdenador() {}
12 };
13
14 #endif

```

```

1  #ifndef ORDENADOR_H
2  #define ORDENADOR_H
3
4  #include "IOrdenador.h"
5
6  template<typename T>
7  class Ordenador : public IOrdenador<T>{
8  protected:
9     void intercambio(T &a, T &b);
10 };
11
12 #include "ordenador.hpp"
13
14 #endif

```

```

1  #ifndef ORDENADOR_HPP
2  #define ORDENADOR_HPP
3
4  template<typename T>
5  void Ordenador<T>::intercambio(T &a, T &b){
6     T aux = a;
7     a = b;
8     b = aux;
9 }
10
11 #endif

```

```

1  #ifndef QUICK_H
2  #define QUICK_H
3
4  #include <vector>
5  #include "Ordenador.h"
6  using namespace std;
7
8  template<typename T>
9  class Quick : public Ordenador<T> {
10 public:
11     void ordenar(vector<T>& n) override;
12
13 private:
14     int particion(vector<T>& n, int inicio, int final);
15     void quicksort(vector<T>& n, int inicio, int final);
16 };
17
18 #include "Quick.hpp"
19
20 #endif

```

```

1  #ifndef QUICK_HPP
2  #define QUICK_HPP
3
4  template<typename T>
5  void Quick<T>::ordenar(vector<T>& n) {
6     if (n.empty()) return;
7     quicksort(n, 0, n.size() - 1);
8 }
9
10 template<typename T>
11 int Quick<T>::particion(vector<T>& n, int inicio, int final) {
12     T pivote = n[final];
13     int p = inicio - 1;
14
15     for (int i = inicio; i < final; i++) {
16         if (n[i] < pivote) {
17             p++;
18             this->intercambio(n[i], n[p]);
19         }
20     }
21
22     p++;
23     this->intercambio(n[p], n[final]);
24
25     return p;
26 }
27
28
29 template<typename T>
30 void Quick<T>::quicksort(vector<T>& n, int inicio, int final) {
31     if (inicio < final) {
32         int pivote = particion(n, inicio, final);
33
34         quicksort(n, inicio, pivote - 1);
35         quicksort(n, pivote + 1, final);
36     }
37 }
38
39
40 #endif

```

```

1  #ifndef UTILIDADES_H
2  #define UTILIDADES_H
3
4  #include <vector>
5  #include <string>
6  using namespace std;
7
8  template<typename T>
9  class Utilidades{
10 public:
11     static void leer(vector<T>& v);
12     static void leerString(vector<T>& v);
13     static void imprimir(const vector<T>& v);
14 };
15
16 #include "Utilidades.hpp"
17
18 #endif

```

```

1  #ifndef PERSONA_H
2  #define PERSONA_H
3
4  #include <iostream>
5  #include <string>
6  using namespace std;
7
8  class persona{
9  private:
10     string nombre;
11     int edad;
12
13 public:
14     static int cr;
15
16     persona();
17
18     string getNombre() const;
19     int getEdad() const;
20
21     void setNombre(const string&);
22     void setEdad(int);
23
24     bool operator<(const persona&) const;
25     bool operator>(const persona&) const;
26
27     friend ostream& operator<<(ostream&, const persona&);
28     friend istream& operator>>(istream&, persona&);
29 };
30
31 #endif

```

```

1  #ifndef UTILIDADES_TPP
2  #define UTILIDADES_TPP
3
4  #include <iostream>
5  #include <vector>
6  #include <string>
7  using namespace std;
8
9  template<typename T>
10 void Utilidades<T>::leer(vector<T>& v){
11     cout << "Introduzca valores\n";
12     for(int i = 0; i < v.size(); i++){
13         cin >> v[i];
14     }
15 }
16
17 template<typename T>
18 void Utilidades<T>::leerString(vector<T>& v){
19     cout << "Introduzca valores\n";
20     for(size_t i = 0; i < v.size(); i++){
21         getline(cin >> ws, v[i]);
22     }
23 }
24
25 template<typename T>
26 void Utilidades<T>::imprimir(const vector<T>& v){
27     for(int i = 0; i < v.size(); i++){
28         cout << v[i] << " ";
29     }
30     cout << endl;
31 }
32

```

```

1  #include "persona.h"
2
3  int persona::cr = 1;
4
5  persona::persona(){
6     nombre="";
7     edad=0;
8 }
9
10 string persona::getNombre() const{return nombre;}
11 int persona::getEdad() const{return edad;}
12
13 void persona::setNombre(const string& n){nombre=n;}
14 void persona::setEdad(int e){ if(e>=0) edad=e; }
15
16 ostream& operator<<(ostream& os, const persona &p){
17     os<<"Nombre: "<<p.getNombre()<<" Edad: "<<p.getEdad();
18     return os;
19 }
20
21 istream& operator>>(istream& is, persona &p){
22     string n;
23     int e;
24
25     cout<<"Nombre: ";
26     if(is.peek()=='\n') is.ignore();
27     getline(is,n);
28

```

```

29     cout<<"Edad: ";
30     is>>e;
31
32     p.setNombre(n);
33     p.setEdad(e);
34     return is;
35 }
36
37 bool persona::operator>(const persona& o) const{
38     if(cr==1) return nombre > o.nombre;
39     return edad > o.edad;
40 }
41
42 bool persona::operator<(const persona& o) const{
43     if(cr==1) return nombre < o.nombre;
44     return edad < o.edad;
45 }

```

Práctica 19 – Ordenamiento Indirecto (Burbuja)

Funcionamiento

Ordena mediante índices o punteros en lugar de mover directamente los datos. Mantiene un arreglo de referencias.

Errores

- Confusión entre datos reales y referencias.
- Errores al actualizar punteros.
- Desincronización entre índices y datos.

```
1  #include <iostream>
2  #include <vector>
3
4  #include "BurbujaI.h"
5  #include "UtilidadesI.h"
6  #include "persona.h"
7  using namespace std;
8
9  int main(){
10     int CA, TV;
11
12     cout << "Cantidad de valores\n";
13     cin >> CA;
14
15     cout << "Tipo Entero(1)/Float(2)/String(3)/Persona(4)\n";
16     cin >> TV;
17
18     if(TV == 1){
19         vector<int> n(CA);
20         vector<int*> p(CA);
21
22         Utilidades<int>::leer(n);
23         Utilidades<int>::direcciones(p, n);
24
25         IOordenador<int>* ord = new Burbuja<int>();
26
27         Utilidades<int>::imprimir(p);
28         ord->ordenar(p);
29         Utilidades<int>::imprimir(p);
30
31         delete ord;
```

```
32     }
33     else if(TV == 2){
34         vector<float> n(CA);
35         vector<float*> p(CA);
36
37         Utilidades<float>::leer(n);
38         Utilidades<float>::direcciones(p, n);
39
40         IOordenador<float>* ord = new Burbuja<float>();
41
42         Utilidades<float>::imprimir(p);
43         ord->ordenar(p);
44         Utilidades<float>::imprimir(p);
45
46         delete ord;
47     }
48     else if(TV == 3){
49         vector<string> n(CA);
50         vector<string*> p(CA);
51
52         Utilidades<string>::leerString(n);
53         Utilidades<string>::direcciones(p, n);
54
55         IOordenador<string>* ord = new Burbuja<string>();
56
57         Utilidades<string>::imprimir(p);
58         ord->ordenar(p);
59         Utilidades<string>::imprimir(p);
60
61         delete ord;
```

```
62     }
63     else if(TV == 4){
64         vector<persona> n(CA);
65         vector<persona*> p(CA);
66
67         cout << "Ordenar por Nombre(1)/Edad(2)\n";
68         cin >> persona::cr;
69
70         Utilidades<persona>::leer(n);
71         Utilidades<persona>::direcciones(p, n);
72
73         IOordenador<persona>* ord = new Burbuja<persona>();
74
75         Utilidades<persona>::imprimir(p);
76         ord->ordenar(p);
77         Utilidades<persona>::imprimir(p);
78
79         delete ord;
80     }
81
82     return 0;
83 }
```

```

1  #ifndef BURBUJAI_H
2  #define BURBUJAI_H
3
4  #include "OrdenadorI.h"
5  #include <vector>
6  using namespace std;
7
8  template<typename T>
9  class Burbuja : public Ordenador<T>{
10 public:
11     void ordenar(vector<T*>& datos) override;
12 };
13
14 #include "BurbujaI.tpp"
15 #endif

```

```

1  #ifndef BURBUJAI_TPP
2  #define BURBUJAI_TPP
3
4  template<typename T>
5  void Burbuja<T>::ordenar(vector<T*>& datos){
6      for(size_t i = 0; i < datos.size(); i++){
7          for(size_t j = 0; j < datos.size() - 1 - i; j++){
8              if(*datos[j] > *datos[j + 1]){
9                  this->intercambio(datos[j], datos[j + 1]);
10             }
11         }
12     }
13 }
14
15 #endif

```

```

1  #ifndef IORDENADORI_H
2  #define IORDENADORI_H
3
4  #include <vector>
5  using namespace std;
6
7  template<typename T>
8  class IOrdenador{
9  public:
10     virtual void ordenar(vector<T*>& datos) = 0;
11     virtual ~IOrdenador() {}
12 };
13
14 #endif

```

```

1  #ifndef ORDENADOR_H
2  #define ORDENADOR_H
3
4  #include "IOrdenadorI.h"
5
6  template<typename T>
7  class Ordenador : public IOrdenador<T>{
8  protected:
9      void intercambio(T*& a, T*& b);
10 };
11
12 #include "OrdenadorI.tpp"
13 #endif

```

```

1  #ifndef ORDENADORI_TPP
2  #define ORDENADORI_TPP
3
4  template<typename T>
5  void Ordenador<T>::intercambio(T*& a, T*& b){
6      T* aux = a;
7      a = b;
8      b = aux;
9  }
10
11 #endif

```

```

1  #ifndef UTILIDADES_I_H
2  #define UTILIDADES_I_H
3
4  #include <vector>
5  #include <string>
6  using namespace std;
7
8  template<typename T>
9  class Utilidades{
10 public:
11     static void leer(vector<T>& v);
12     static void leerString(vector<T>& v);
13     static void imprimir(const vector<T*>& v);
14     static void direcciones(vector<T*>&, vector<T>& n);
15 };
16
17 #include "UtilidadesI.tpp"
18 #endif

```

```

1  #ifndef UTILIDADES_TP
2  #define UTILIDADES_TP
3
4  #include <iostream>
5  #include <vector>
6  #include <string>
7
8  using namespace std;
9
10 template<typename T>
11 void Utilidades<T>::leer(vector<T> &v){
12     cout << "Introduzca valores\n";
13     for(size_t i = 0; i < v.size(); i++){
14         cin >> v[i];
15     }
16 }
17
18 template<typename T>
19 void Utilidades<T>::leerString(vector<T> &v){
20     cout << "Introduzca valores\n";
21     for(size_t i = 0; i < v.size(); i++){
22         getline(cin >> ws, v[i]);
23     }
24 }
25
26 template<typename T>
27 void Utilidades<T>::imprimir(const vector<T> &v){
28     for(size_t i = 0; i < v.size(); i++){
29         cout << *v[i] << " ";
30     }
31     cout << endl;
32 }
33
34 template<typename T>
35 void Utilidades<T>::direcciones(vector<T*> &p, vector<T> &n){
36     for(size_t i = 0; i < p.size(); i++){
37         p[i] = &n[i];
38     }
39 }
40
41 #endif

```

```

1  #ifndef PERSONA_H
2  #define PERSONA_H
3
4  #include <iostream>
5  #include <string>
6  using namespace std;
7
8  class persona{
9  private:
10     string nombre;
11     int edad;
12
13 public:
14     static int cr;
15
16     persona();
17
18     string getNombre() const;
19     int getEdad() const;
20
21     void setNombre(const string&);
22     void setEdad(int);
23
24     bool operator>(const persona&);
25
26     friend ostream& operator<<(ostream&, const persona&);
27     friend istream& operator>>(istream&, persona&);
28 };
29
30 #endif

```

```

1  #include "persona.h"
2
3  int persona::cr = 1;
4
5  persona::persona(){
6     nombre = "";
7     edad = 0;
8 }
9
10 string persona::getNombre() const { return nombre; }
11 int persona::getEdad() const { return edad; }
12
13 void persona::setNombre(const string& n){ nombre = n; }
14 void persona::setEdad(int e){ if(e >= 0) edad = e; }
15
16 ostream& operator<<(ostream& os, const persona& p){
17     os << "Nombre: " << p.getNombre()
18     << " Edad: " << p.getEdad();
19     return os;
20 }
21
22 istream& operator>>(istream& is, persona& p){
23     string n;
24     int e;
25
26     cout << "Nombre: ";
27     getline(is >> ws, n);
28
29     cout << "Edad: ";

```

```

30     is >> e;
31
32     p.setNombre(n);
33     p.setEdad(e);
34
35     return is;
36 }
37
38 bool persona::operator>(const persona& o){
39     if(cr == 1) return nombre > o.nombre;
40     return edad > o.edad;
41 }

```

Práctica 20 – Ordenamiento Indirecto (Merge Sort)

Funcionamiento

Aplica merge sort pero usando referencias (punteros o índices), evitando mover datos directamente.

Errores

- Manejo incorrecto de referencias en la fusión.
- Uso incorrecto de memoria auxiliar.

```
1  #include <iostream>
2  #include <vector>
3
4  #include "MergeI.h"
5  #include "UtilidadesI.h"
6  #include "persona.h"
7
8  using namespace std;
9  int main(){
10     int CA, TV;
11
12     cout << "Cantidad de valores\n";
13     cin >> CA;
14
15     cout << "Tipo Entero(1)/Float(2)/String(3)/Persona(4)\n";
16     cin >> TV;
17
18     if(TV == 1){
19         vector<int> n(CA);
20         vector<int*> p(CA);
21
22         Utilidades<int>::leer(n);
23         Utilidades<int>::direcciones(p, n);
24
25         IOordenador<int>* ord = new Merge<int>();
26
27         Utilidades<int>::imprimir(p);
28         ord->ordenar(p);
29         Utilidades<int>::imprimir(p);
30
31         delete ord;
```

```
32     }
33     else if(TV == 2){
34         vector<float> n(CA);
35         vector<float*> p(CA);
36
37         Utilidades<float>::leer(n);
38         Utilidades<float>::direcciones(p, n);
39
40         IOordenador<float>* ord = new Merge<float>();
41
42         Utilidades<float>::imprimir(p);
43         ord->ordenar(p);
44         Utilidades<float>::imprimir(p);
45
46         delete ord;
47     }
48     else if(TV == 3){
49         vector<string> n(CA);
50         vector<string*> p(CA);
51
52         Utilidades<string>::leerString(n);
53         Utilidades<string>::direcciones(p, n);
54
55         IOordenador<string>* ord = new Merge<string>();
56
57         Utilidades<string>::imprimir(p);
58         ord->ordenar(p);
59         Utilidades<string>::imprimir(p);
60
61         delete ord;
```

```
62     }
63     else if(TV == 4){
64         vector<persona> n(CA);
65         vector<persona*> p(CA);
66
67         cout << "Ordenar por Nombre(1)/Edad(2)\n";
68         cin >> persona::cr;
69
70         Utilidades<persona>::leer(n);
71         Utilidades<persona>::direcciones(p, n);
72
73         IOordenador<persona>* ord = new Merge<persona>();
74
75         Utilidades<persona>::imprimir(p);
76         ord->ordenar(p);
77         Utilidades<persona>::imprimir(p);
78
79         delete ord;
80     }
81
82     return 0;
83 }
```

```

1  #ifndef IORDENADORI_H
2  #define IORDENADORI_H
3
4  #include <vector>
5  using namespace std;
6
7  template<typename T>
8  class IOrdenador{
9  public:
10     virtual void ordenar(vector<T*>& datos) = 0;
11     virtual ~IOrdenador() {}
12 };
13
14 #endif

```

```

1  #ifndef MERGEI_H
2  #define MERGEI_H
3
4  #include <vector>
5  #include "OrdenadorI.h"
6
7  using namespace std;
8
9  template<typename T>
10 class Merge : public Ordenador<T>{
11 public:
12     void ordenar(vector<T*>& v) override;
13
14 private:
15     void mergeSort(vector<T*>& v, int izq, int der);
16     void merge(vector<T*>& v, int izq, int mid, int der);
17 };
18
19 #include "MergeI.tpp"
20 #endif

```

```

1  #ifndef MERGEI_TPP
2  #define MERGEI_TPP
3
4  template<typename T>
5  void Merge<T>::ordenar(vector<T*>& v){
6      if(v.empty()) return;
7      mergeSort(v, 0, v.size() - 1);
8  }
9
10 template<typename T>
11 void Merge<T>::mergeSort(vector<T*>& v, int izq, int der){
12     if(izq >= der) return;
13
14     int mid = izq + (der - izq) / 2;
15
16     mergeSort(v, izq, mid);
17     mergeSort(v, mid + 1, der);
18     merge(v, izq, mid, der);
19 }
20
21 template<typename T>
22 void Merge<T>::merge(vector<T*>& v, int izq, int mid, int der){
23     vector<T*> aux;
24
25     int i = izq;
26     int j = mid + 1;
27
28     while(i <= mid && j <= der){
29         if(*v[i] < *v[j]){
30             aux.push_back(v[i]);
31             i++;
32         }else{
33             aux.push_back(v[j]);
34             j++;
35         }
36     }
37
38     while(i <= mid){
39         aux.push_back(v[i]);
40         i++;
41     }
42
43     while(j <= der){
44         aux.push_back(v[j]);
45         j++;
46     }
47
48     for(size_t k = 0; k < aux.size(); k++){
49         v[izq + k] = aux[k];
50     }
51 }
52
53 #endif

```

```

1  #ifndef ORDENADOR_H
2  #define ORDENADOR_H
3
4  #include "IOrdenadorI.h"
5
6  template<typename T>
7  class Ordenador : public IOrdenador<T>{
8  protected:
9      void intercambio(T*& a, T*& b);
10 };
11
12 #include "OrdenadorI.tpp"
13 #endif

```

```

1  #ifndef ORDENADORI_TPP
2  #define ORDENADORI_TPP
3
4  template<typename T>
5  void Ordenador<T>::intercambio(T*& a, T*& b){
6      T* aux = a;
7      a = b;
8      b = aux;
9  }
10
11 #endif

```



```

1  #ifndef UTILIDADESI_H
2  #define UTILIDADESI_H
3
4  #include <vector>
5  #include <string>
6  using namespace std;
7
8  template<typename T>
9  class Utilidades{
10 public:
11     static void leer(vector<T>& v);
12     static void leerString(vector<T>& v);
13     static void imprimir(const vector<T*>& v);
14     static void direcciones(vector<T*>&, vector<T>& n);
15 };
16
17 #include "UtilidadesI.tpp"
18 #endif

```

```

1  #ifndef UTILIDADESI_TPP
2  #define UTILIDADESI_TPP
3
4  #include <iostream>
5  #include <vector>
6  #include <string>
7
8  using namespace std;
9
10 template<typename T>
11 void Utilidades<T>::leer(vector<T>& v){
12     cout << "Introduzca valores\n";
13     for(size_t i = 0; i < v.size(); i++){
14         cin >> v[i];
15     }
16 }
17
18 template<typename T>
19 void Utilidades<T>::leerString(vector<T>& v){
20     cout << "Introduzca valores\n";
21     for(size_t i = 0; i < v.size(); i++){
22         getline(cin >> ws, v[i]);
23     }
24 }
25
26 template<typename T>
27 void Utilidades<T>::imprimir(const vector<T*>& v){
28     for(size_t i = 0; i < v.size(); i++){
29         cout << *v[i] << " ";
30     }
31     cout << endl;
32 }
33
34 template<typename T>
35 void Utilidades<T>::direcciones(vector<T*>& p, vector<T>& n){
36     for(size_t i = 0; i < p.size(); i++){
37         p[i] = &n[i];
38     }
39 }
40
41 #endif

```

```

1  #ifndef PERSONA_H
2  #define PERSONA_H
3
4  #include <iostream>
5  #include <string>
6  using namespace std;
7
8  class persona{
9  private:
10     string nombre;
11     int edad;
12
13 public:
14     static int cr;
15
16     persona();
17
18     string getNombre() const;
19     int getEdad() const;
20
21     void setNombre(const string&);
22     void setEdad(int);
23
24     bool operator<(const persona&) const;
25     bool operator>(const persona&) const;
26
27     friend ostream& operator<<(ostream&, const persona&);
28     friend istream& operator>>(istream&, persona&);
29 };
30
31 #endif

```

```

1  #include "persona.h"
2
3  int persona::cr = 1;
4
5  persona::persona(){
6     nombre="";
7     edad=0;
8 }
9
10 string persona::getNombre() const{return nombre;}
11 int persona::getEdad() const{return edad;}
12
13 void persona::setNombre(const string& n){nombre=n;}
14 void persona::setEdad(int e){ if(e>=0) edad=e; }
15
16 ostream& operator<<(ostream& os, const persona &p){
17     os<<"Nombre: "<<p.getNombre()<<" Edad: "<<p.getEdad();
18     return os;
19 }
20
21 istream& operator>>(istream& is, persona &p){
22     string n;
23     int e;
24
25     cout<<"Nombre: ";
26     if(is.peek()=='\n') is.ignore();
27     getline(is,n);
28
29     cout<<"Edad: ";
30     is>>e;
31
32     p.setNombre(n);
33     p.setEdad(e);
34     return is;
35 }
36
37 bool persona::operator>(const persona& o) const{
38     if(cr==1) return nombre > o.nombre;
39     return edad > o.edad;
40 }
41
42 bool persona::operator<(const persona& o) const{
43     if(cr==1) return nombre < o.nombre;
44     return edad < o.edad;
45 }

```

Práctica 21 – Ordenamiento Indirecto (Quick Sort)

Funcionamiento

Ordena usando punteros o índices con la lógica de quick sort.

Errores

- Errores en intercambio de referencias.
- Pérdida de correspondencia entre punteros y datos reales.

```
1  #include <iostream>
2  #include <vector>
3
4  #include "QuickI.h"
5  #include "UtilidadesI.h"
6  #include "persona.h"
7
8  using namespace std;
9  int main(){
10     int CA, TV;
11
12     cout << "Cantidad de valores\n";
13     cin >> CA;
14
15     cout << "Tipo Entero(1)/Float(2)/String(3)/Persona(4)\n";
16     cin >> TV;
17
18     if(TV == 1){
19         vector<int> n(CA);
20         vector<int*> p(CA);
21
22         Utilidades<int>::leer(n);
23         Utilidades<int>::direcciones(p, n);
24
25         IOordenador<int>* ord = new Quick<int>();
26
27         Utilidades<int>::imprimir(p);
28         ord->ordenar(p);
29         Utilidades<int>::imprimir(p);
30
31         delete ord;
32     }
```

```
33     else if(TV == 2){
34         vector<float> n(CA);
35         vector<float*> p(CA);
36
37         Utilidades<float>::leer(n);
38         Utilidades<float>::direcciones(p,n);
39
40         IOordenador<float>* ord = new Quick<float>();
41
42         Utilidades<float>::imprimir(p);
43         ord->ordenar(p);
44         Utilidades<float>::imprimir(p);
45
46         delete ord;
47     }
48     else if(TV == 3){
49         vector<string> n(CA);
50         vector<string*> p(CA);
51
52         Utilidades<string>::leerString(n);
53         Utilidades<string>::direcciones(p, n);
54
55         IOordenador<string>* ord = new Quick<string>();
56
57         Utilidades<string>::imprimir(p);
58         ord->ordenar(p);
59         Utilidades<string>::imprimir(p);
60
61         delete ord;
```

```
62     }
63     else if(TV == 4){
64         vector<persona> n(CA);
65         vector<persona*> p(CA);
66
67         cout << "Ordenar por Nombre(1)/Edad(2)\n";
68         cin >> persona::cr;
69
70         Utilidades<persona>::leer(n);
71         Utilidades<persona>::direcciones(p, n);
72
73         IOordenador<persona>* ord = new Quick<persona>();
74
75         Utilidades<persona>::imprimir(p);
76         ord->ordenar(p);
77         Utilidades<persona>::imprimir(p);
78
79         delete ord;
80     }
81
82     return 0;
83 }
```

```

1  #ifndef IORDENADORI_H
2  #define IORDENADORI_H
3
4  #include <vector>
5  using namespace std;
6
7  template<typename T>
8  class IOrdenador{
9  public:
10     virtual void ordenar(vector<T*>& datos) = 0;
11     virtual ~IOrdenador() {}
12 };
13
14 #endif

```

```

1  #ifndef ORDENADOR_H
2  #define ORDENADOR_H
3
4  #include "IOrdenadorI.h"
5
6  template<typename T>
7  class Ordenador : public IOrdenador<T>{
8  protected:
9     void intercambio(T*& a, T*& b);
10 };
11
12 #include "OrdenadorI.tpp"
13 #endif

```

```

1  #ifndef ORDENADORI_TPP
2  #define ORDENADORI_TPP
3
4  template<typename T>
5  void Ordenador<T>::intercambio(T*& a, T*& b){
6     T* aux = a;
7     a = b;
8     b = aux;
9 }
10
11 #endif

```

```

1  #ifndef QUICKI_H
2  #define QUICKI_H
3
4  #include <vector>
5  #include "OrdenadorI.h"
6  using namespace std;
7
8  template<typename T>
9  class Quick : public Ordenador<T> {
10 public:
11     void ordenar(vector<T*>& n) override;
12
13 private:
14     int particion(vector<T*>& n, int inicio, int final);
15     void quicksort(vector<T*>& n, int inicio, int final);
16 };
17
18 #include "QuickI.tpp"
19
20 #endif

```

```

1  #ifndef QUICKI_TPP
2  #define QUICKI_TPP
3
4  template<typename T>
5  void Quick<T>::ordenar(vector<T*>& n) {
6     if (n.empty()) return;
7     quicksort(n, 0, n.size() - 1);
8 }
9
10 template<typename T>
11 int Quick<T>::particion(vector<T*>& n, int inicio, int final) {
12     T* pivote = n[final];
13     int p = inicio - 1;
14
15     for (int i = inicio; i < final; i++) {
16         if (*n[i] < *pivote) {
17             p++;
18             this->intercambio(n[i], n[p]);
19         }
20     }
21
22     p++;
23     this->intercambio(n[p], n[final]);
24
25     return p;
26 }
27
28
29 template<typename T>
30 void Quick<T>::quicksort(vector<T*>& n, int inicio, int final) {
31     if (inicio < final) {
32         int pivote = particion(n, inicio, final);
33
34         quicksort(n, inicio, pivote - 1);
35         quicksort(n, pivote + 1, final);
36     }
37 }
38
39
40 #endif

```

```

1  #ifndef UTILIDADES_I_H
2  #define UTILIDADES_I_H
3
4  #include <vector>
5  #include <string>
6  using namespace std;
7
8  template<typename T>
9  class Utilidades{
10 public:
11     static void leer(vector<T>& v);
12     static void leerString(vector<T>& v);
13     static void imprimir(const vector<T*>& v);
14     static void direcciones(vector<T*>& p, vector<T>& n);
15 };
16
17 #include "UtilidadesI.tpp"
18 #endif

```

```

1  #ifndef UTILIDADES_I_TPP
2  #define UTILIDADES_I_TPP
3
4  #include <iostream>
5  #include <vector>
6  #include <string>
7
8  using namespace std;
9
10 template<typename T>
11 void Utilidades<T>::leer(vector<T>& v){
12     cout << "Introduzca valores\n";
13     for(size_t i = 0; i < v.size(); i++){
14         cin >> v[i];
15     }
16 }
17
18 template<typename T>
19 void Utilidades<T>::leerString(vector<T>& v){
20     cout << "Introduzca valores\n";
21     for(size_t i = 0; i < v.size(); i++){
22         getline(cin >> ws, v[i]);
23     }
24 }
25
26 template<typename T>
27 void Utilidades<T>::imprimir(const vector<T*>& v){
28     for(size_t i = 0; i < v.size(); i++){
29         cout << *v[i] << " ";
30     }
31     cout << endl;
32 }
33
34 template<typename T>
35 void Utilidades<T>::direcciones(vector<T*>& p, vector<T>& n){
36     for(size_t i = 0; i < p.size(); i++){
37         p[i] = &n[i];
38     }
39 }
40
41 #endif

```

```

1  #ifndef PERSONA_H
2  #define PERSONA_H
3
4  #include <iostream>
5  #include <string>
6  using namespace std;
7
8  class persona{
9  private:
10     string nombre;
11     int edad;
12
13 public:
14     static int cr;
15
16     persona();
17
18     string getNombre() const;
19     int getEdad() const;
20
21     void setNombre(const string&);
22     void setEdad(int);
23
24     bool operator<(const persona&) const;
25     bool operator>(const persona&) const;
26
27     friend ostream& operator<<(ostream&, const persona&);
28     friend istream& operator>>(istream&, persona&);
29 };
30
31 #endif

```

```

1  #include "persona.h"
2
3  int persona::cr = 1;
4
5  persona::persona(){
6     nombre="";
7     edad=0;
8 }
9
10 string persona::getNombre() const{return nombre;}
11 int persona::getEdad() const{return edad;}
12
13 void persona::setNombre(const string& n){nombre=n;}
14 void persona::setEdad(int e){ if(e>0) edad=e; }
15
16 ostream& operator<<(ostream& os, const persona& p){
17     os<<"Nombre: "<<p.getNombre()<<" Edad: "<<p.getEdad();
18     return os;
19 }
20
21 istream& operator>>(istream& is, persona& p){
22     string n;
23     int e;
24
25     cout<<"Nombre: ";
26     if(is.peek()=='\n') is.ignore();
27     getline(is,n);
28
29     cout<<"Edad: ";
30     is>>e;
31
32     p.setNombre(n);
33     p.setEdad(e);
34     return is;
35 }
36
37 bool persona::operator>(const persona& o) const{
38     if(cr==1) return nombre > o.nombre;
39     return edad > o.edad;
40 }
41
42 bool persona::operator<(const persona& o) const{
43     if(cr==1) return nombre < o.nombre;
44     return edad < o.edad;
45 }

```

Práctica 1 – Punteros con Clases/Estructuras

Funcionamiento

Uso de punteros o referencias dentro de estructuras o clases para manipular datos indirectamente.

Errores

- Punteros no inicializados.
- Acceso inválido a memoria.

```
1  #include <iostream>
2  #include "ListaPersonas.h"
3  using namespace std;
4
5
6  int main(int argc, char** argv) {
7
8      Persona personas[5];
9
10     ListaPersonas lista(personas, 5);
11
12     lista.capturar();
13     lista.mostrar();
14
15     return 0;
16 }
```

```

1  #ifndef LISTAPERSONAS_H
2  #define LISTAPERSONAS_H
3
4  #include <string>
5  using namespace std;
6
7  struct Persona {
8      string nombre;
9      string ap;
10     string am;
11     string genero;
12     int edad;
13 };
14
15 class ListaPersonas
16 {
17     private:
18         Persona* ptr;    // puntero al arreglo
19         int tam;
20
21     public:
22         ListaPersonas(Persona* arreglo, int tamano);
23         void capturar();
24         void mostrar();
25 };
26
27 #endif

```

```

1  #include "ListaPersonas.h"
2  #include <iostream>
3  using namespace std;
4
5  ListaPersonas::ListaPersonas(Persona* arreglo, int tamano){
6      ptr = arreglo;
7      tam = tamano;
8  }
9
10 //ListaPersonas::~ListaPersonas(){}
11
12 //Capturar datos
13
14 void ListaPersonas::capturar() {
15     for (int i = 0; i < tam; i++) {
16         cout << "\nPersona " << i + 1 << endl;
17
18         cout << "Nombre: ";
19         cin >> (ptr + i)->nombre;
20
21         cout << "Apellido paterno: ";
22         cin >> (ptr + i)->ap;
23
24         cout << "Apellido materno: ";
25         cin >> (ptr + i)->am;
26
27         cout << "Genero: ";
28         cin >> (ptr + i)->genero;
29
30         cout << "Edad: ";
31         cin >> (ptr + i)->edad;
32     }
33 }
34
35 //Mostrar datos
36
37 void ListaPersonas::mostrar() {
38     cout << "\n=== LISTA DE PERSONAS ===\n";
39
40     for (int i = 0; i < tam; i++) {
41         cout << "\nPersona " << i + 1 << endl;
42
43         cout << "Nombre completo: "
44             << (ptr + i)->nombre << " "
45             << (ptr + i)->ap << " "
46             << (ptr + i)->am << endl;
47
48         cout << "Genero: " << (ptr + i)->genero << endl;
49         cout << "Edad: " << (ptr + i)->edad << endl;
50     }
51 }

```

Práctica 3 – Pila Estática

Funcionamiento

Implementa una pila (LIFO) usando un arreglo fijo.

Errores

- Manejo incorrecto del índice tope.

```
1  #include "Pila.h"
2  #include "PilaT.h"
3  #include <iostream>
4  #include <string>
5
6  using namespace std;
7
8  int main()
9  {
10     char r = ' ';
11     while(r != 'x')
12     {
13         cout << "Introduzca el tipo de dato: " << endl;
14         cout << "1. int" << endl;
15         cout << "2. float" << endl;
16         cout << "3. string" << endl;
17         cout << "0 introduzca 'x' para finalizar" << endl;
18         cin >> r;
19         if(r == '1') // int
20         {
21             PilaT<int> p;
22             while(r != 'f')
23             {
24                 cout << "1. pop()" << endl;
25                 cout << "2. push()" << endl;
26                 cout << "3. top()" << endl;
27                 cout << "Introduzca la accion a realizar ('f' si desea finalizar y usar otro tipo de dato): ";
28                 cin >> r;
29                 switch(r)
30                 {
31                     case '1':
32                         cout << "Se elimino: " << p.pop() << endl;
33                         break;
34                     case '2':
35                         int temp;
36                         cout << "Introduzca el elemento que sera introducido a la pila: ";
37                         cin >> temp;
38                         p.push(temp);
39                         cout << "Se agrego: " << temp << endl;
40                         break;
41                     case '3':
42                         cout << "El ultimo elemento es: " << p.top() << endl;
43                         break;
44                 }
45             }
46         }
47         if(r == '2') // float
48         {
49             PilaT<float> p;
50             while(r != 'f')
51             {
52                 cout << "1. pop()" << endl;
53                 cout << "2. push()" << endl;
54                 cout << "3. top()" << endl;
55                 cout << "Introduzca la accion a realizar ('f' si desea finalizar y usar otro tipo de dato): ";
56                 cin >> r;
57                 switch(r)
58                 {
59                     case '1':
60                         cout << "Se elimino: " << p.pop() << endl;
```

```

61         break;
62     case '2':
63         float temp;
64         cout << "Introduzca el elemento que sera introducido a la pila: ";
65         cin >> temp;
66         p.push(temp);
67         cout << "Se agrego: " << temp << endl;
68         break;
69     case '3':
70         cout << "El ultimo elemento es: " << p.top() << endl;
71         break;
72     }
73 }
74 }
75 if(r == '3') // string
76 {
77     PilaT<string> p;
78     string temp;
79     while(r != 'f')
80     {
81         cout << "1. pop()" << endl;
82         cout << "2. push()" << endl;
83         cout << "3. top()" << endl;
84         cout << "Introduzca la accion a realizar ('f' si desea finalizar y usar otro tipo de dato): ";
85         cin >> r;
86         switch(r)
87         {
88             case '1':
89                 cout << "Se elimino: " << p.pop() << endl;
90                 break;
91             case '3':
92                 cout << "El ultimo elemento es: " << p.top() << endl;
93                 break;
94             case '2':
95                 cout << "Introduzca la cadena que sera introducido a la pila (sin espacios): ";
96                 cin >> temp;
97                 p.push(temp);
98                 cout << "Se agrego: " << temp << endl;
99                 break;
100         }
101     }
102 }
103 }
104 return 0;
105 }

```

```

1  #ifndef NODO_H
2  #define NODO_H
3
4  template <typename T>
5  class Nodo
6  {
7  public:
8      Nodo();
9      Nodo(const T& d);
10     ~Nodo();
11     T getDato();
12     void setDato(const T& d);
13
14     protected:
15
16     private:
17         T dato;
18     };
19
20     #include "../src/Nodo.hpp"
21
22     #endif // NODO_H

```



```

1  #ifndef PILA_H
2  #define PILA_H
3
4  ✓ class Pila
5  {
6  public:
7      Pila();
8      virtual ~Pila();
9
10     virtual bool vacia() = 0;
11     virtual bool llena() = 0;
12
13 protected:
14     int tope;
15 private:
16
17 };
18
19 #endif // PILA_H

```

```

1  #ifndef PILAT_H
2  #define PILAT_H
3
4  #include "Nodo.h"
5  #include "Pila.h"
6
7  template <typename T>
8  ✓ class PilaT: public Pila
9  {
10 public:
11     PilaT();
12     virtual ~PilaT();
13
14     bool vacia() override;
15     bool llena() override;
16
17     T pop();
18     void push(const T&);
19     T top();
20
21 protected:
22
23 private:
24     static const int TAM = 32;
25     Nodo<T> datos[TAM];
26 };
27
28 #include "../src/PilaT.tpp"
29
30 #endif // PILAT_H

```

```

1  #ifndef A_H
2  #define A_H
3
4
5  ✓ class a
6  {
7      public:
8          a();
9          virtual ~a();
10
11      protected:
12          int hola;
13      private:
14  };
15
16 #endif // A_H

```

```

1  #ifndef NODO_H
2
3  template <typename T>
4  Nodo<T>::Nodo(): dato(T()) {}
5
6  template <typename T>
7  Nodo<T>::Nodo(const T& d):dato(d) {}
8
9  template <typename T>
10  Nodo<T>::~~Nodo() {}
11
12  template <typename T>
13  T Nodo<T>::getDato()
14  {
15      return dato;
16  }
17
18  template <typename T>
19  void Nodo<T>::setDato(const T& d)
20  {
21      dato = d;
22  }
23
24  #endif // NODO_H

```

```

1  #include "Pila.h"
2
3  Pila::Pila(): tope(-1) {}
4
5  Pila::~~Pila() {}

```

```

1  #ifndef PILAT_H
2  #include <iostream>
3  using namespace std;
4
5  template <typename T>
6  PilaT<T>::PilaT()
7  {
8      this->tope = -1;
9  }
10
11  template <typename T>
12  PilaT<T>::~~PilaT() {}
13
14  template <typename T>
15  bool PilaT<T>::vacia()
16  {
17      return this->tope == -1;
18  }
19
20  template <typename T>
21  bool PilaT<T>::llena()
22  {
23      return this->tope == TAM - 1;
24  }
25
26  template <typename T>
27  void PilaT<T>::push(const T& valor)
28  {
29      if(llena())
30      {
31          cerr<<"Error: La pila esta llena." <<endl;

```

```

32      return;
33  }
34      this->tope++;
35      datos[this->tope].setDato(valor);
36  }
37
38  template <typename T>
39  T PilaT<T>::pop()
40  {
41      if(vacia())
42      {
43          cerr<<"Error: La pila esta vacia." <<endl;
44          return T(); // Valor por defecto
45      }
46      T temp = datos[this->tope].getDato();
47      datos[this->tope].setDato(T());
48      this->tope--;
49
50      return temp;
51  }
52
53  template <typename T>
54  T PilaT<T>::top()
55  {
56      if(vacia())
57      {
58          cerr<<"Error: La pila esta vacia." <<endl;
59          return T();
60      }
61      else
62      {
63          return datos[this->tope].getDato();
64      }
65  }
66
67  #endif // PILAT_H

```

Práctica 4 – Cola Estática

Funcionamiento

Implementa una cola (FIFO) usando arreglo.

Errores

- Mala gestión de índices circular.

```
1  #include "ColaT.h"
2  #include <iostream>
3  #include <string>
4
5  using namespace std;
6
7  int main()
8  {
9      char r = ' ';
10     while(r != 'x')
11     {
12         cout << "Introduzca el tipo de dato: " << endl;
13         cout << "1. int" << endl;
14         cout << "2. float" << endl;
15         cout << "3. string" << endl;
16         cout << "0 introduzca 'x' para finalizar" << endl;
17         cin >> r;
18         if(r == '1') // int
19         {
20             ColaT<int> p;
21             while(r != 'f')
22             {
23                 cout << "1. dequeue()" << endl;
24                 cout << "2. enqueue()" << endl;
25                 cout << "3. first()" << endl;
26                 cout << "Introduzca la accion a realizar ('f' si desea finalizar y usar otro tipo de dato): ";
27                 cin >> r;
28                 switch(r)
29                 {
30                     case '1':
31                         cout << "Se elimino: " << p.dequeue() << endl;
32                         break;
33                     case '2':
34                         int temp;
35                         cout << "Introduzca el elemento que sera introducido a la Cola: ";
36                         cin >> temp;
37                         p.enqueue(temp);
38                         cout << "Se agrego: " << temp << endl;
39                         break;
40                     case '3':
41                         cout << "El primer elemento es: " << p.first() << endl;
42                         break;
43                 }
44             }
45         }
46     }
47     if(r == '2') // float
48     {
49         ColaT<float> p;
50         while(r != 'f')
51         {
52             cout << "1. dequeue()" << endl;
53             cout << "2. enqueue()" << endl;
54             cout << "3. first()" << endl;
55             cout << "Introduzca la accion a realizar ('f' si desea finalizar y usar otro tipo de dato): ";
56             cin >> r;
57             switch(r)
58             {
59                 case '1':
60                     cout << "Se elimino: " << p.dequeue() << endl;
```

```

61         break;
62     case '2':
63         float temp;
64         cout << "Introduzca el elemento que sera introducido a la Cola: ";
65         cin >> temp;
66         p.enqueue(temp);
67         cout << "Se agrego: " << temp << endl;
68         break;
69     case '3':
70         cout << "El primer elemento es: " << p.first() << endl;
71         break;
72     }
73 }
74 }
75 if(r == '3') // string
76 {
77     ColaT<string> p;
78     string temp;
79     while(r != 'f')
80     {
81         cout << "1. dequeue()" << endl;
82         cout << "2. enqueue()" << endl;
83         cout << "3. first()" << endl;
84         cout << "Introduzca la accion a realizar ('f' si desea finalizar y usar otro tipo de dato): ";
85         cin >> r;
86         switch(r)
87         {
88             case '1':
89                 cout << "Se elimino: " << p.dequeue() << endl;
90                 break;
91             case '3':
92                 cout << "El primer elemento es: " << p.first() << endl;
93                 break;
94             case '2':
95                 cout << "Introduzca la cadena que sera introducido a la Cola (sin espacios): ";
96                 cin >> temp;
97                 p.enqueue(temp);
98                 cout << "Se agrego: " << temp << endl;
99                 break;
100         }
101     }
102 }
103 }
104 return 0;
105 }

```

```

1  #ifndef NODO_H
2  #define NODO_H
3
4  template <typename T>
5  class Nodo
6  {
7  public:
8      Nodo();
9      Nodo(const T& d);
10     ~Nodo();
11     T getDato();
12     void setDato(const T& d);
13
14     protected:
15
16     private:
17         T dato;
18     };
19
20     #include "../src/Nodo.tpp"
21
22     #endif // NODO_H

```

```

1  #ifndef COLA_H
2  #define COLA_H
3
4  class Cola
5  {
6  public:
7      Cola();
8      virtual ~Cola();
9
10     virtual bool vacia() = 0;
11     virtual bool llena() = 0;
12
13     protected:
14         int frente, final;
15     private:
16
17     };
18
19     #endif // COLA_H

```

```

1  #ifndef COLAT_H
2  #define COLAT_H
3
4  #include "Cola.h"
5  #include "Nodo.h"
6
7  template <typename T>
8  class ColaT:public Cola
9  {
10 public:
11     ColaT();
12     ~ColaT();
13
14     bool llena() override;
15     bool vacia() override;
16
17     T dequeue();
18     void enqueue(const T& valor);
19     T first();
20
21     protected:
22
23     private:
24         static const int TAM = 32;
25         Nodo<T> datos[TAM];
26     };
27
28     #include "../src/ColaT.tpp"
29
30     #endif // COLAT_H

```

```

1  #include "Cola.h"
2
3  Cola::Cola(): frente(-1), final(-1) {}
4
5  Cola::~Cola() {}

```

```

1  #ifndef NODO_H
2
3  template <typename T>
4  Nodo<T>::Nodo(): dato(T()) {}
5
6  template <typename T>
7  Nodo<T>::Nodo(const T& d):dato(d) {}
8
9  template <typename T>
10 Nodo<T>::~~Nodo() {}
11
12 template <typename T>
13 T Nodo<T>::getDato()
14 {
15     return dato;
16 }
17
18 template <typename T>
19 void Nodo<T>::setDato(const T& d)
20 {
21     dato = d;
22 }
23
24 #endif // NODO_H

```

```

1  #ifndef COLAT_H
2
3  #include <iostream>
4  using namespace std;
5
6  template<typename T>
7  ColaT<T>::ColaT():Cola(){}
8
9  template<typename T>
10 ColaT<T>::~~ColaT(){}
11
12 template <typename T>
13 bool ColaT<T>::vacía()
14 {
15     return this->frente == -1;
16 }
17
18 template <typename T>
19 bool ColaT<T>::llena()
20 {
21     return this->final == TAM - 1;
22 }
23
24 template <typename T>
25 void ColaT<T>::enqueue(const T& valor)
26 {
27     if(llena())
28     {
29         cerr<<"Error: La cola esta llena." <<endl;
30         return;
31     }

```

```

32     if(vacía())
33     {
34         this->frente = 0;
35     }
36     this->final++;
37     datos[this->final].setDato(valor);
38 }
39
40 template <typename T>
41 T ColaT<T>::dequeue()
42 {
43     if(vacía())
44     {
45         cerr<<"Error: La cola esta vacia." <<endl;
46         return T();
47     }
48
49     T temp = datos[this->frente].getDato();
50     datos[this->frente].setDato(T());
51
52     if(this->frente == this->final) // solo habia un elemento
53     {
54         this->frente = -1;
55         this->final = -1;
56     }
57     else // mas de un elemento
58     {
59         this->frente++;
60     }
61
62     return temp;
63 }
64
65 template <typename T>
66 T ColaT<T>::first()
67 {
68     if(vacía())
69     {
70         cerr<<"Error: La cola esta vacia." <<endl;
71         return T();
72     }
73     return datos[this->frente].getDato();
74 }
75
76 #endif

```

Práctica 5 – Lista Estática

Funcionamiento

Lista implementada con arreglo.

Errores

- Inserciones fuera de rango.
- Desplazamientos incorrectos.

```
1  #include "ListaI.h"
2  #include <iostream>
3  #include <string>
4
5  using namespace std;
6
7  int main()
8  {
9      char r = ' ';
10     int i = 0;
11     while(r != 'x')
12     {
13         cout << "Introduzca el tipo de dato: " << endl;
14         cout << "1. int" << endl;
15         cout << "2. float" << endl;
16         cout << "3. string" << endl;
17         cout << "0 introduzca 'x' para finalizar" << endl;
18         cin >> r;
19         if(r == '1')// int
20         {
21             ListaI<int> p;
22             while(r != 'f')
23             {
24                 cout << "1. eliminar()" << endl;
25                 cout << "2. insertar()" << endl;
26                 cout << "3. obtener()" << endl;
27                 cout << "Introduzca la accion a realizar ('f' si desea finalizar y usar otro tipo de dato): ";
28                 cin >> r;
29                 switch(r)
30                 {
31                     case '1':
32                         cout << "Introduzca el indice del elemento a eliminar: ";
```

```

33         cin >> i;
34         cout << "Se elimino: "<< p.obtener(i) << " en el indice "<< i << endl;
35         p.eliminar(i);
36         cout << "Longitud: " << p.longitud() << endl;
37         break;
38     case '2':
39         int temp;
40         cout << "Introduzca el elemento que sera introducido a la lista: ";
41         cin >> temp;
42         cout << "Introduzca el indice en donde sera introducido: ";
43         cin >> i;
44         p.insertar(temp, i);
45         cout << "Se agrego: " << temp << " en el indice " << i << endl;
46         cout << "Longitud: " << p.longitud() << endl;
47         break;
48     case '3':
49         cout << "Introduzca el indice del elemento a mostrar: ";
50         cin >> i;
51         cout << "El elemento en "<< i << " es: " << p.obtener(i) << endl;
52         cout << "Longitud: " << p.longitud() << endl;
53         break;
54     }
55 }
56 }
57 if(r == '2') // float
58 {
59     ListaT<float> p;
60     int i;
61     while(r != 'f')

```

```

62 {
63     cout << "1. eliminar()" << endl;
64     cout << "2. insertar()" << endl;
65     cout << "3. obtener()" << endl;
66     cout << "Introduzca la accion a realizar ('f' si desea finalizar y usar otro tipo de dato): ";
67     cin >> r;
68     switch(r)
69     {
70     case '1':
71         cout << "Introduzca el indice del elemento a eliminar: ";
72         cin >> i;
73         cout << "Se elimino: "<< p.obtener(i) << " en el indice "<< i << endl;
74         p.eliminar(i);
75         cout << "Longitud: " << p.longitud() << endl;
76         break;
77     case '2':
78         float temp;
79         cout << "Introduzca el elemento que sera introducido a la lista: ";
80         cin >> temp;
81         cout << "Introduzca el indice en donde sera introducido: ";
82         cin >> i;
83         p.insertar(temp, i);
84         cout << "Se agrego: " << temp << " en el indice " << i << endl;
85         cout << "Longitud: " << p.longitud() << endl;
86         break;
87     case '3':
88         cout << "Introduzca el indice del elemento a mostrar: ";
89         cin >> i;
90         cout << "El elemento en "<< i << " es: " << p.obtener(i) << endl;
91         cout << "Longitud: " << p.longitud() << endl;
92         break;
93     }
94 }
95 }
96 if(r == '3') // string
97 {
98     ListaT<string> p;
99     int i;
100    string temp;
101    while(r != 'f')
102    {
103        cout << "1. eliminar()" << endl;
104        cout << "2. insertar()" << endl;
105        cout << "3. obtener()" << endl;
106        cout << "Introduzca la accion a realizar ('f' si desea finalizar y usar otro tipo de dato): ";
107        cin >> r;
108        switch(r)
109        {
110        case '1':
111            cout << "Introduzca el indice del elemento a eliminar: ";
112            cin >> i;
113            cout << "Se elimino: "<< p.obtener(i) << " en el indice "<< i << endl;
114            p.eliminar(i);
115            cout << "Longitud: " << p.longitud() << endl;
116            break;
117        case '2':
118            cout << "Introduzca el elemento que sera introducido a la lista: ";
119            cin >> temp;

```

```

120         cout << "Introduzca el indice en donde sera introducido: ";
121         cin >> i;
122         p.insertar(temp, i);
123         cout << "Se agrego: " << temp << " en el indice " << i << endl;
124         cout << "Longitud: " << p.longitud() << endl;
125         break;
126     case '3':
127         cout << "Introduzca el indice del elemento a mostrar: ";
128         cin >> i;
129         cout << "El elemento en "<< i << " es: " << p.obtener(i) << endl;
130         cout << "Longitud: " << p.longitud() << endl;
131         break;
132     }
133 }
134 }
135 }
136 return 0;
137 }

```



```

1  #ifndef LISTA_H
2  #define LISTA_H
3
4  class Lista
5  {
6  public:
7      Lista();
8      ~Lista();
9
10     virtual bool vacia() = 0;
11     virtual bool llena() = 0;
12
13
14     int longitud() const;
15
16 protected:
17     int ultimo;
18 private:
19
20 };
21
22 #endif // LISTA_H

```

```

1  #ifndef LISTAT_H
2  #define LISTAT_H
3  #include "Lista.h"
4  #include "Nodo.h"
5
6  template <typename T>
7  class ListaT:public Lista
8  {
9  public:
10     ListaT();
11     virtual ~ListaT();
12
13     bool vacia() override;
14     bool llena() override;
15
16
17     void insertar(T valor, int pos);
18     void eliminar(int pos);
19     T obtener(int pos);
20
21
22 protected:
23
24 private:
25     static const int TAM = 32;
26     Nodo<T> datos[TAM];
27 };
28
29 #include "../src/ListaT.tpp"
30
31 #endif // LISTAT_H

```

```

1  #ifndef NODO_H
2  #define NODO_H
3
4  template <typename T>
5  class Nodo
6  {
7  public:
8      Nodo();
9      Nodo(const T& d);
10     ~Nodo();
11     T getDato();
12     void setDato(const T& d);
13
14 protected:
15
16 private:
17     T dato;
18 };
19
20 #include "../src/Nodo.tpp"
21
22 #endif // NODO_H

```

```

1  #include "Lista.h"
2
3  Lista::Lista():ultimo(-1) {}
4
5  Lista::~Lista() {}
6
7  int Lista::longitud() const
8  {
9      return ultimo+1;
10 }

```

```

1  #ifdef NODO_H
2
3  template <typename T>
4  Nodo<T>::Nodo(): dato(T()) {}
5
6  template <typename T>
7  Nodo<T>::Nodo(const T& d):dato(d) {}
8
9  template <typename T>
10 Nodo<T>::~~Nodo() {}
11
12 template <typename T>
13 T Nodo<T>::getDato()
14 {
15     return dato;
16 }
17
18 template <typename T>
19 void Nodo<T>::setDato(const T& d)
20 {
21     dato = d;
22 }
23
24 #endif // NODO_H

```

```

1  #ifdef LISTAT_H
2
3  #include <iostream>
4  using namespace std;
5  template <typename T>
6  Lista<T>::Lista():Lista() {}
7
8  template <typename T>
9  Lista<T>::~~Lista() {}
10
11
12 template <typename T>
13 bool Lista<T>::vacia()
14 {
15     return this->ultimo == -1;
16 }
17
18 template <typename T>
19 bool Lista<T>::llena()
20 {
21     return this->ultimo == TAM - 1;
22 }
23
24 template <typename T>
25 void Lista<T>::insertar(T valor, int pos)
26 {
27     if(llena() || pos < 0 || pos > longitud())
28     {
29         cerr<<"Error: La lista esta vacia." <<endl;
30         return;
31     }
32     for(int i = this->ultimo; i>= pos; i--)

```

```

33     {
34         datos[i+1].setDato(datos[i].getDato());
35     }
36     datos[pos].setDato(valor);
37     this->ultimo++;
38 }
39
40 template <typename T>
41 void Lista<T>::eliminar(int pos)
42 {
43     if(vacia() || pos < 0 || pos > this->ultimo)
44     {
45         cerr<<"Error: La lista esta vacia." <<endl;
46         return;
47     }
48     for(int i = pos; i < this->ultimo; i++)
49     {
50         datos[i].setDato(datos[i+1].getDato());
51     }
52     datos[this->ultimo].setDato(T());
53     this->ultimo--;
54 }
55
56 template <typename T>
57 T Lista<T>::obtener(int pos)
58 {
59     if(vacia() || pos < 0 || pos > this->ultimo)
60     {
61         cerr<<"Error: La lista esta vacia." <<endl;
62         return T();
63     }
64
65     return datos[pos].getDato();
66 }
67
68 #endif // LISTAT_H

```

Práctica 6 – Pila Dinámica (Punteros)

Funcionamiento

Pila implementada con nodos enlazados usando memoria dinámica.

Errores

- Punteros nulos.
- Mala liberación de memoria.

```
1  #include "Pila.h"
2  #include <iostream>
3  #include <string>
4
5  using namespace std;
6
7  ✓ int main()
8  {
9      char r = ' ';
10     while(r != 'x')
11     {
12         cout << "Introduzca el tipo de dato: " << endl;
13         cout << "1. int" << endl;
14         cout << "2. float" << endl;
15         cout << "3. string" << endl;
16         cout << "0 introduzca 'x' para finalizar" << endl;
17         cin >> r;
18         if(r == '1')// int
19         {
20             Pila<int> p;
21             while(r != 'f')
22             {
23                 cout << "1. pop()" << endl;
24                 cout << "2. push()" << endl;
25                 cout << "3. peek()" << endl;
26                 cout << "Introduzca la accion a realizar ('f' si desea finalizar y usar otro tipo de dato): ";
27                 cin >> r;
28                 switch(r)
29                 {
30                     case '1':
31                         cout << "Se elimino: " << p.pop() << endl;
32                         break;
```

```

33         case '2':
34             int temp;
35             cout << "Introduzca el elemento que sera introducido a la pila: ";
36             cin >> temp;
37             p.push(temp);
38             cout << "Se agrego: " << temp << endl;
39             break;
40         case '3':
41             cout << "El ultimo elemento es: " << p.peek() << endl;
42             break;
43     }
44 }
45 }
46 if(r == '2') // float
47 {
48     Pila<float> p;
49     while(r != 'f')
50     {
51         cout << "1. pop()" << endl;
52         cout << "2. push()" << endl;
53         cout << "3. peek()" << endl;
54         cout << "Introduzca la accion a realizar ('f' si desea finalizar y usar otro tipo de dato): ";
55         cin >> r;
56         switch(r)
57         {
58             case '1':
59                 cout << "Se elimino: " << p.pop() << endl;
60                 break;
61             case '2':
62                 float temp;
63                 cout << "Introduzca el elemento que sera introducido a la pila: ";
64                 cin >> temp;
65                 p.push(temp);
66                 cout << "Se agrego: " << temp << endl;
67                 break;
68             case '3':
69                 cout << "El ultimo elemento es: " << p.peek() << endl;
70                 break;
71         }
72     }
73 }
74 if(r == '3') // string
75 {
76     Pila<string> p;
77     string temp;
78     while(r != 'f')
79     {
80         cout << "1. pop()" << endl;
81         cout << "2. push()" << endl;
82         cout << "3. peek()" << endl;
83         cout << "Introduzca la accion a realizar ('f' si desea finalizar y usar otro tipo de dato): ";
84         cin >> r;
85         switch(r)
86         {
87             case '1':
88                 cout << "Se elimino: " << p.pop() << endl;
89                 break;
90             case '3':
91                 cout << "El ultimo elemento es: " << p.peek() << endl;
92                 break;
93             case '2':
94                 cout << "Introduzca la cadena que sera introducido a la pila (sin espacios): ";
95                 cin >> temp;
96                 p.push(temp);
97                 cout << "Se agrego: " << temp << endl;
98                 break;
99         }
100     }
101 }
102 }
103 return 0;
104 }

```

```

1  #ifndef COLECCION_H
2  #define COLECCION_H
3
4
5  class Coleccion
6  {
7  public:
8      Coleccion();
9      virtual ~Coleccion();
10
11      virtual bool vacia() = 0;
12      virtual int longitud() = 0;
13
14  protected:
15      int tam;
16  private:
17
18  };
19
20 #endif // COLECCION_H

```

```

1  #ifndef PILA_H
2  #define PILA_H
3
4  #include "Coleccion.h"
5  #include "Nodo.h"
6
7  template <typename T>
8  class Pila: public Coleccion
9  {
10 public:
11     Pila();
12     ~Pila();
13
14     bool vacia() override;
15     int longitud() override;
16
17     bool push(const T& elem);
18     T pop();
19     const T& peek();
20
21 protected:
22
23 private:
24     Nodo<T>* tope;
25 };
26
27 #include "../src/Pila.tpp"
28
29 #endif // PILA_H

```

```

1  #ifndef NODO_H
2  #define NODO_H
3
4  template <typename T>
5  class Nodo
6  {
7  public:
8      T dato;
9      Nodo<T>* siguiente;
10
11      Nodo(const T& valor);
12      virtual ~Nodo();
13
14  protected:
15
16  private:
17  };
18
19 #include "../src/Nodo.tpp"
20
21 #endif // NODO_H

```

```

1  #include "Coleccion.h"
2
3  Coleccion::Coleccion():tam(0) {}
4
5  Coleccion::~~Coleccion() {}

```

```

1  #ifndef NODO_H
2
3  template <typename T>
4  Nodo<T>::Nodo(const T& valor):dato(valor), siguiente(nullptr) {}
5
6  template <typename T>
7  Nodo<T>::~~Nodo() {}
8
9  #endif // NODO_H

```

```

1  #ifndef PILA_H
2
3  template <typename T>
4  Pila<T>::Pila():Coleccion(),tope(nullptr) {}
5
6  template <typename T>
7  Pila<T>::~~Pila()
8  {
9      while(!vacía())
10     {
11         pop();
12     }
13 }
14
15 template <typename T>
16 bool Pila<T>::vacía()
17 {
18     return tope == nullptr;
19 }
20
21 template <typename T>
22 int Pila<T>::longitud()
23 {
24     return tam;
25 }
26
27 template <typename T>
28 bool Pila<T>::push(const T& elem)
29 {
30     Nodo<T>* temp = new Nodo<T>(elem);
31     temp->siguiente = tope;
32     tope = temp;

```

```

33     tam++;
34     return true;
35 }
36
37 template <typename T>
38 T Pila<T>::pop()
39 {
40     if(vacía())
41     {
42         return T();
43     }
44
45     T temp = tope->dato;
46     Nodo<T>* aux = tope;
47     tope = tope->siguiente;
48
49     delete aux;
50     tam--;
51     return temp;
52 }
53
54 template <typename T>
55 const T& Pila<T>::peek()
56 {
57     if(vacía())
58     {
59         static T nulo = T();
60         return nulo;
61     }
62
63     return tope->dato;
64 }
65 #endif // PILA_H

```

Práctica 7 – Cola Dinámica (Punteros)

Funcionamiento

Cola usando lista enlazada.

Errores

- Mal manejo de frente y final.

```
1  #include "Cola.h"
2  #include <iostream>
3  #include <string>
4
5  using namespace std;
6
7  int main()
8  {
9      char r = ' ';
10     while(r != 'x')
11     {
12         cout << "Introduzca el tipo de dato: " << endl;
13         cout << "1. int" << endl;
14         cout << "2. float" << endl;
15         cout << "3. string" << endl;
16         cout << "0 introduzca 'x' para finalizar" << endl;
17         cin >> r;
18         if(r == '1')// int
19         {
20             Cola<int> p;
21             while(r != 'f')
22             {
23                 cout << "1. dequeue()" << endl;
24                 cout << "2. enqueue()" << endl;
25                 cout << "3. front()" << endl;
26                 cout << "Introduzca la accion a realizar ('f' si desea finalizar y usar otro tipo de dato): ";
27                 cin >> r;
28                 switch(r)
29                 {
30                     case '1':
31                         cout << "Se elimino: " << p.dequeue() << endl;
32                         break;
```

```

32         break;
33     case '2':
34         int temp;
35         cout << "Introduzca el elemento que sera introducido a la Cola: ";
36         cin >> temp;
37         p.enqueue(temp);
38         cout << "Se agrego: " << temp << endl;
39
40         break;
41     case '3':
42         cout << "El primer elemento es: " << p.front() << endl;
43         break;
44     }
45 }
46 }
47 if(r == '2') // float
48 {
49     Cola<float> p;
50     while(r != 'f')
51     {
52         cout << "1. dequeue()" << endl;
53         cout << "2. enqueue()" << endl;
54         cout << "3. front()" << endl;
55         cout << "Introduzca la accion a realizar ('f' si desea finalizar y usar otro tipo de dato): ";
56         cin >> r;
57         switch(r)
58         {
59             case '1':
60                 cout << "Se elimino: " << p.dequeue() << endl;
61                 break;
62             case '2':
63                 float temp;
64                 cout << "Introduzca el elemento que sera introducido a la Cola: ";
65                 cin >> temp;
66                 p.enqueue(temp);
67                 cout << "Se agrego: " << temp << endl;
68                 break;
69             case '3':
70                 cout << "El primer elemento es: " << p.front() << endl;
71                 break;
72         }
73     }
74 }
75 if(r == '3') // string
76 {
77     Cola<string> p;
78     string temp;
79     while(r != 'f')
80     {
81         cout << "1. dequeue()" << endl;
82         cout << "2. enqueue()" << endl;
83         cout << "3. front()" << endl;
84         cout << "Introduzca la accion a realizar ('f' si desea finalizar y usar otro tipo de dato): ";
85         cin >> r;
86         switch(r)
87         {
88             case '1':
89                 cout << "Se elimino: " << p.dequeue() << endl;
90                 break;
91             case '3':
92                 cout << "El primer elemento es: " << p.front() << endl;
93                 break;
94             case '2':
95                 cout << "Introduzca la cadena que sera introducido a la Cola (sin espacios): ";
96                 cin >> temp;
97                 p.enqueue(temp);
98                 cout << "Se agrego: " << temp << endl;
99                 break;
100         }
101     }
102 }
103 }
104 return 0;
105 }

```



```

1  #ifndef COLA_H
2  #define COLA_H
3
4  #include "Coleccion.h"
5  #include "Nodo.h"
6
7  template <typename T>
8  class Cola: public Coleccion
9  {
10 public:
11     Cola();
12     ~Cola();
13
14     bool vacia() override;
15     int longitud() override;
16
17     bool enqueue(const T& elem);
18     T dequeue();
19     const T& front();
20
21 protected:
22
23 private:
24     Nodo<T>* frente;
25     Nodo<T>* final;
26 };
27
28 #include "../src/Cola.tpp"
29
30 #endif // COLA_H

```

```

1  #ifndef COLECCION_H
2  #define COLECCION_H
3
4
5  class Coleccion
6  {
7  public:
8      Coleccion();
9      virtual ~Coleccion();
10
11      virtual bool vacia() = 0;
12      virtual int longitud() = 0;
13
14 protected:
15     int tam;
16 private:
17
18 };
19
20 #endif // COLECCION_H

```

```

1  #ifndef NODO_H
2  #define NODO_H
3
4  template <typename T>
5  class Nodo
6  {
7  public:
8      T dato;
9      Nodo<T>* siguiente;
10
11      Nodo(const T& valor);
12      virtual ~Nodo();
13
14 protected:
15
16 private:
17 };
18
19 #include "../src/Nodo.tpp"
20
21 #endif // NODO_H

```

```

1  #include "Coleccion.h"
2
3  Coleccion::Coleccion():tam(0) {}
4
5  Coleccion::~Coleccion() {}

```

```

1  #ifndef NODO_H
2
3  template <typename T>
4  Nodo<T>::Nodo(const T& valor):dato(valor), siguiente(nullptr) {}
5
6  template <typename T>
7  Nodo<T>::~Nodo() {}
8
9  #endif // NODO_H

```

```

1  #ifndef COLA_H
2
3  template <typename T>
4  Cola<T>::Cola():Coleccion(),frente(nullptr),final(nullptr) {}
5
6  template <typename T>
7  Cola<T>::~Cola()
8  {
9      while(!vacía())
10     {
11         dequeue();
12     }
13 }
14
15 template <typename T>
16 bool Cola<T>::vacía()
17 {
18     return frente == nullptr;
19 }
20
21 template <typename T>
22 int Cola<T>::longitud()
23 {
24     return this->tam;
25 }
26
27 template <typename T>
28 bool Cola<T>::enqueue(const T& elem)
29 {
30     Nodo<T>* temp = new Nodo<T>(elem);
31     if(vacía())

```

```

32     {
33         frente = temp;
34         final = temp;
35     }
36     else
37     {
38         final->siguiente = temp;
39         final = temp;
40     }
41     this->tam++;
42     return true;
43 }
44
45 template <typename T>
46 T Cola<T>::dequeue()
47 {
48     if(vacía())
49     {
50         return T();
51     }
52
53     T temp = frente->dato;
54     Nodo<T>* aux = frente;
55     frente = frente->siguiente;
56
57     if(frente == nullptr)
58     {
59         final = nullptr;
60     }
61
62     delete aux;
63     this->tam--;
64     return temp;
65 }
66
67 template <typename T>
68 const T& Cola<T>::front()
69 {
70     if(vacía())
71     {
72         static T nulo = T();
73         return nulo;
74     }
75
76     return frente->dato;
77 }
78 #endif // COLA_H

```

Práctica 7 bis – Lista Dinámica

Funcionamiento

Lista enlazada dinámica con nodos.

Errores

- Recorridos incorrectos.

```
1  #include "Lista.h"
2  #include <iostream>
3  #include <string>
4
5  using namespace std;
6
7  int main()
8  {
9      char r = ' ';
10     int i = 0;
11     while(r != 'x')
12     {
13         cout << "Introduzca el tipo de dato: " << endl;
14         cout << "1. int" << endl;
15         cout << "2. float" << endl;
16         cout << "3. string" << endl;
17         cout << "0 introduzca 'x' para finalizar" << endl;
18         cin >> r;
19         if(r == '1')// int
20         {
21             Lista<int> p;
22             while(r != 'f')
23             {
24                 cout << "1. remove()" << endl;
25                 cout << "2. add()" << endl;
26                 cout << "3. get()" << endl;
27                 cout << "Introduzca la accion a realizar ('f' si desea finalizar y usar otro tipo de dato): ";
28                 cin >> r;
29                 switch(r)
30                 {
31                     case '1':
32                         cout << "Introduzca el indice del elemento a remove: ";
```

```

32     cout << "Introduzca el índice del elemento a remove: ";
33     cin >> i;
34     cout << "Se elimino: "<< p.remove(i) << " en el índice "<< i << endl;
35     cout << "Longitud: " << p.longitud() << endl;
36     break;
37 case '2':
38     int temp;
39     cout << "Introduzca el elemento que sera introducido a la lista: ";
40     cin >> temp;
41     cout << "Introduzca el índice en donde sera introducido: ";
42     cin >> i;
43     if(p.add(temp, i))
44     {
45         cout << "Se agrego: " << temp << " en el índice " << i << endl;
46         cout << "Longitud: " << p.longitud() << endl;
47     }
48     else
49     {
50         cout << "Hubo un error" << endl;
51     }
52     break;
53 case '3':
54     cout << "Introduzca el índice del elemento a mostrar: ";
55     cin >> i;
56     cout << "El elemento en "<< i << " es: " << p.get(i) << endl;
57     cout << "Longitud: " << p.longitud() << endl;
58     break;
59 }
60 }
61 }
62 if(r == '2') // float
63 {
64     Lista<float> p;
65     int i;
66     while(r != 'f')
67     {
68         cout << "1. remove()" << endl;
69         cout << "2. add()" << endl;
70         cout << "3. get()" << endl;
71         cout << "Introduzca la accion a realizar ('f' si desea finalizar y usar otro tipo de dato): ";
72         cin >> r;
73         switch(r)
74         {
75             case '1':
76                 cout << "Introduzca el índice del elemento a remove: ";
77                 cin >> i;
78                 cout << "Se elimino: "<< p.remove(i) << " en el índice "<< i << endl;
79                 cout << "Longitud: " << p.longitud() << endl;
80                 break;
81             case '2':
82                 float temp;
83                 cout << "Introduzca el elemento que sera introducido a la lista: ";
84                 cin >> temp;
85                 cout << "Introduzca el índice en donde sera introducido: ";
86                 cin >> i;
87                 if(p.add(temp, i))
88                 {
89                     cout << "Se agrego: " << temp << " en el índice " << i << endl;
90                     cout << "Longitud: " << p.longitud() << endl;
91                 }
92                 else
93                 {
94                     cout << "Hubo un error" << endl;
95                 }
96                 break;
97             case '3':
98                 cout << "Introduzca el índice del elemento a mostrar: ";
99                 cin >> i;
100                cout << "El elemento en "<< i << " es: " << p.get(i) << endl;
101                cout << "Longitud: " << p.longitud() << endl;
102                break;
103            }
104        }
105    }
106    if(r == '3') // string
107    {
108        Lista<string> p;
109        int i;
110        string temp;
111        while(r != 'f')
112        {
113            cout << "1. remove()" << endl;
114            cout << "2. add()" << endl;
115            cout << "3. get()" << endl;
116            cout << "Introduzca la accion a realizar ('f' si desea finalizar y usar otro tipo de dato): ";
117            cin >> r;
118            switch(r)

```

```

119        {
120            case '1':
121                cout << "Introduzca el índice del elemento a remove: ";
122                cin >> i;
123                cout << "Se elimino: "<< p.remove(i) << " en el índice "<< i << endl;
124                cout << "Longitud: " << p.longitud() << endl;
125                break;
126            case '2':
127                cout << "Introduzca el elemento que sera introducido a la lista: ";
128                cin >> temp;
129                cout << "Introduzca el índice en donde sera introducido: ";
130                cin >> i;
131                if(p.add(temp, i))
132                {
133                    cout << "Se agrego: " << temp << " en el índice " << i << endl;
134                    cout << "Longitud: " << p.longitud() << endl;
135                }
136                else
137                {
138                    cout << "Hubo un error" << endl;
139                }
140                break;
141            case '3':
142                cout << "Introduzca el índice del elemento a mostrar: ";
143                cin >> i;
144                cout << "El elemento en "<< i << " es: " << p.get(i) << endl;
145                cout << "Longitud: " << p.longitud() << endl;
146                break;
147            }
148        }
149    }
150    }
151    return 0;
152 }

```

```

1  #ifndef COLECCION_H
2  #define COLECCION_H
3
4
5  ✓ class Coleccion
6  {
7  public:
8      Coleccion();
9      virtual ~Coleccion();
10
11      virtual bool vacia() = 0;
12      virtual int longitud() = 0;
13
14  protected:
15      int tam;
16  private:
17
18  };
19
20 #endif // COLECCION_H

```

```

1  #ifndef NODO_H
2  #define NODO_H
3
4  template <typename T>
5  ✓ class Nodo
6  {
7  public:
8      T dato;
9      Nodo<T>* siguiente;
10
11      Nodo(const T& valor);
12      virtual ~Nodo();
13
14  protected:
15
16  private:
17  };
18
19 #include "../src/Nodo.hpp"
20
21 #endif // NODO_H

```

```

1  #ifndef LISTA_H
2  #define LISTA_H
3
4  #include "Nodo.h"
5  #include "Coleccion.h"
6
7  template <typename T>
8  ✓ class Lista: public Coleccion
9  {
10 public:
11     Lista();
12     ~Lista();
13
14     bool vacia() override;
15     int longitud() override;
16
17     bool add(const T& elem, int pos);
18
19     bool set(int pos, const T& elem);
20
21     T remove(int pos);
22
23     const T& get(int pos);
24
25 protected:
26
27 private:
28     Nodo<T>* inicio;
29 };
30
31 #include "../src/Lista.hpp"
32

```

```

1      #include "Coleccion.h"
2
3      Coleccion::Coleccion():tam(0) {}
4
5      Coleccion::~~Coleccion() {}

```

```

1      #ifndef NODO_H
2
3      template <typename T>
4      Nodo<T>::Nodo(const T& valor):dato(valor), siguiente(nullptr) {}
5
6      template <typename T>
7      Nodo<T>::~~Nodo() {}
8
9      #endif // NODO_H

```

```

1      #ifndef LISTA_H
2
3      template <typename T>
4      Lista<T>::Lista():Coleccion(), inicio(nullptr) {}
5
6      template <typename T>
7      Lista<T>::~~Lista()
8      {
9          while(!vacia())
10         {
11             remove(tam-1);
12         }
13     }
14
15     template <typename T>
16     bool Lista<T>::vacia()
17     {
18         return inicio == nullptr;
19     }
20
21     template <typename T>
22     int Lista<T>::longitud()
23     {
24         return tam;
25     }
26
27     template <typename T>
28     bool Lista<T>::add(const T& elem, int pos)
29     {
30         if(pos < 0 || pos > tam)
31         {
32             return false;
33         }
34
35         Nodo<T>* temp = new Nodo(elem);
36
37         if(pos == 0)
38         {
39             temp->siguiente = inicio;
40             inicio = temp;
41             tam++;
42             return true;
43         }
44
45         Nodo<T>* actual = inicio;
46         for(int i = 0; i < pos - 1; i++)
47         {
48             actual = actual->siguiente;
49         }
50         temp->siguiente = actual->siguiente;
51         actual->siguiente = temp;
52
53         tam++;
54         return true;
55     }
56
57     template <typename T>
58     bool Lista<T>::set(int pos, const T& elem)
59     {
60         if(pos < 0 || pos > tam)
61         {
62             return false;

```

```

63         }
64         Nodo<T>* actual = inicio;
65         for(int i = 0; i < pos; i++)
66         {
67             actual = actual->siguiente;
68         }
69         actual->dato = elem;
70         return true;
71     }
72
73     template <typename T>
74     T Lista<T>::remove(int pos)
75     {
76         if(vacia() || pos < 0 || pos > tam - 1)
77         {
78             return T();
79         }
80         Nodo<T>* aBorrar = nullptr;
81         if(pos == 0)
82         {
83             aBorrar = inicio;
84             inicio = nullptr;
85         }
86         else
87         {
88             Nodo<T>* anterior = inicio;
89             for(int i = 0; i < pos - 1; i++)
90             {
91                 anterior = anterior->siguiente;
92             }

```

```

93             Nodo<T>* aBorrar = anterior->siguiente;
94             anterior->siguiente = aBorrar->siguiente;
95         }
96
97         T dato = aBorrar->dato;
98         delete aBorrar;
99         tam--;
100        return dato;
101    }
102
103    template<typename T>
104    const T& Lista<T>::get(int pos)
105    {
106        if(pos < 0 || pos > tam)
107        {
108            static T nulo = T();
109            return nulo;
110        }
111        Nodo<T>* actual = inicio;
112        for(int i = 0; i < pos; i++)
113        {
114            actual = actual->siguiente;
115        }
116        return actual->dato;
117    }
118
119    #endif // LISTA_H

```

Práctica 8 – Pila con Librerías/Clases

Funcionamiento

Uso de estructuras ya implementadas en el IDE o librerías estándar.

Errores

- Uso incorrecto de métodos.

```
1  #include <iostream>
2  #include <stack>
3  #include <limits>
4  using namespace std;
5
6  class persona {
7  private:
8      string nombre;
9      int edad;
10
11  public:
12      persona() : nombre(""), edad(0) {}
13      persona(string n, int e) {
14
15          nombre = n;
16          edad = e;
17      }
18      void setNombre(string n) { nombre = n; }
19      void setEdad(int e) { edad = e; }
20
21      string getNombre() const { return nombre; }
22      int getEdad() const { return edad; }
23  };
24
25  ostream& operator<<(ostream &os, const persona &p) {
26      os << "Nombre: " << p.getNombre()
27          << " Edad: " << p.getEdad();
28      return os;
29  }
30  istream& operator>>(istream &is, persona &p) {
31      string nombre;
```

```

32     int edad;
33
34     is.ignore(numeric_limits<streamsize>::max(), '\n');
35
36     cout << "Nombre: ";
37     getline(is, nombre);
38
39     cout << "Edad: ";
40     is >> edad;
41
42     p.setNombre(nombre);
43     p.setEdad(edad);
44
45     return is;
46 }
47 template<typename T>
48 void mostrar(stack<T> pila) {
49     cout<<"Pila actual"<<endl;
50     while (!pila.empty()) {
51         cout << pila.top() << endl;
52         pila.pop();
53     }
54 }
55 template<typename T>
56 void menu(){
57     int op;
58     stack<T> pila;
59     T t;
60     int c=1;
61     while(c!=0){
62         cout<<"Operacion 1.push 2.pop 3.leer"<<endl;
63         cin>>op;
64         switch(op){
65             case 1:
66                 cout<<"Introduzca los dato"<<endl;
67                 cin>> t;
68                 pila.push(t);
69                 break;
70             case 2:
71                 if(!pila.empty()) {
72                     cout<<"Valor eliminado: "<<pila.top()<<endl;
73                     pila.pop();
74                 }
75                 else{
76                     cout<<"Pila vacia"<<endl;
77                 }
78                 break;
79             case 3:
80                 mostrar(pila);
81                 break;
82             }
83         cout<<"Continuar? 1/0"<<endl;
84         cin>>c;
85     }
86
87 }
88
89 int main() {
90     int op;

```

```

91     cout<<"Tipo de dato 1.Int 2.Float 3.String 4.Class"<<endl;
92     cin>>op;
93     switch(op){
94         case 1:
95             menu<int>();
96             break;
97         case 2:
98             menu<float>();
99             break;
100        case 3:
101            menu<string>();
102            break;
103        case 4:
104            menu<persona>();
105            break;
106        default:
107            cout<<"Valor invalido"<<endl;
108        }
109        return 0;
110    }

```


Práctica 9 – Cola con Librerías/Clases

Funcionamiento

Uso de colas predefinidas.

Errores

- Mal uso de funciones enqueue/dequeue.

```
1  #include <iostream>
2  #include <queue>
3  #include <limits>
4  using namespace std;
5  class persona {
6  private:
7      string nombre;
8      int edad;
9
10 public:
11     persona() : nombre(""), edad(0) {}
12     persona(string n, int e) : nombre(n), edad(e) {}
13
14     void setNombre(string n) { nombre = n; }
15     void setEdad(int e) { edad = e; }
16
17     string getNombre() const { return nombre; }
18     int getEdad() const { return edad; }
19 };
20
21 // SALIDA
22 ostream& operator<<(ostream &os, const persona &p) {
23     os << "Nombre: " << p.getNombre()
24         << " Edad: " << p.getEdad();
25     return os;
26 }
27
28 // ENTRADA
29 istream& operator>>(istream &is, persona &p) {
30     string nombre;
31     int edad;
32     is.ignore(numeric_limits<streamsize>::max(), '\n');
```

```

33     cout << "Nombre: ";
34     getline(is, nombre);
35     cout << "Edad: ";
36     is >> edad;
37     p.setNombre(nombre);
38     p.setEdad(edad);
39
40     return is;
41 }
42
43 // MOSTRAR
44 template<typename T>
45 void mostrar(queue<T> cola) {
46     cout << "\nCola actual:\n";
47     while (!cola.empty()) {
48         cout << cola.front() << endl;
49         cola.pop();
50     }
51 }
52
53 // CONTAR
54 template<typename T>
55 int contar(queue<T> cola) {
56     int c = 0;
57     while (!cola.empty()) {
58         cola.pop();
59         c++;
60     }
61     return c;
62 }
63
64 // =====
65 // VERSION SIN if constexpr
66 // =====
67
68 // Para tipos normales
69 template<typename T>
70 void leerDato(T &t) {
71     cin >> t;
72 }
73
74 // Especialización para string
75 template<>
76 void leerDato<string>(string &t) {
77     cin.ignore(numeric_limits<streamsize>::max(), '\n');
78     getline(cin, t);
79 }
80
81 // =====
82 // MENU
83 // =====
84 template<typename T>
85 void menu() {
86     queue<T> cola;
87     int op, c = 1;
88     T t;
89
90     while (c != 0) {
91         cout << "\n--- MENU COLA ---\n";
92         cout << "1. Enqueue\n2. Dequeue\n3. Mostrar\n4. Frente\n5. Contar\n";

```

```

93     cout << "Seleccione: ";
94     cin >> op;
95
96     switch (op) {
97     case 1:
98         cout << "Ingrese dato:\n";
99         leerDato(t);
100        cola.push(t);
101        break;
102
103     case 2:
104         if (!cola.empty()) {
105             cout << "Eliminado: " << cola.front() << endl;
106             cola.pop();
107         } else {
108             cout << "Cola vacia\n";
109         }
110         break;
111
112     case 3:
113         mostrar(cola);
114         break;
115
116     case 4:
117         if (!cola.empty()) {
118             cout << "Frente: " << cola.front() << endl;
119         } else {
120             cout << "Cola vacia\n";
121         }
122         break;
123
124     case 5:
125         cout << "Total: " << contar(cola) << endl;
126         break;
127
128     default:
129         cout << "Opcion invalida\n";
130     }
131
132     cout << "Continuar? 1 = si / 0 = no: ";
133     cin >> c;
134 }
135 }
136
137 // MAIN
138 int main() {
139     int op;
140
141     cout << "--- TIPO DE DATO ---\n";
142     cout << "1. Int\n2. Float\n3. String\n4. Persona\n";
143     cin >> op;
144
145     switch (op) {
146     case 1: menu<int>(); break;
147     case 2: menu<float>(); break;
148     case 3: menu<string>(); break;
149     case 4: menu<persona>(); break;
150     default: cout << "Opcion invalida\n";
151     }
152
153     return 0;
154 }

```

Práctica 10 – Lista con Librerías/Clases

Funcionamiento

Uso de listas dinámicas de librerías.

Errores

- Confusión entre tipos de listas.
- Errores al iterar o modificar.

```
1  #include <iostream>
2  #include <list>
3  #include <limits>
4  using namespace std;
5  class persona {
6  private:
7      string nombre;
8      int edad;
9
10 public:
11     persona() : nombre(""), edad(0) {}
12     persona(string n, int e) : nombre(n), edad(e) {}
13
14     void setNombre(string n) { nombre = n; }
15     void setEdad(int e) { edad = e; }
16
17     string getNombre() const { return nombre; }
18     int getEdad() const { return edad; }
19 };
20 ostream& operator<<(ostream &os, const persona &p) {
21     os << "Nombre: " << p.getNombre()
22     << " Edad: " << p.getEdad();
23     return os;
24 }
25 istream& operator>>(istream &is, persona &p) {
26     string nombre;
27     int edad;
28
29     is.ignore(numeric_limits<streamsize>::max(), '\n');
30
31     cout << "Nombre: ";
32     getline(is, nombre);
```

```

33
34     cout << "Edad: ";
35     is >> edad;
36
37     p.setNombre(nombre);
38     p.setEdad(edad);
39
40     return is;
41 }
42 template<typename T>
43 void mostrar(list<T> lista) {
44     cout << "\nLista actual:\n";
45     for (auto it = lista.begin(); it != lista.end(); ++it) {
46         cout << *it << endl;
47     }
48 }
49 template<typename T>
50 void buscar(list<T> lista, T valor) {
51     for (auto it = lista.begin(); it != lista.end(); ++it) {
52         if (*it == valor) {
53             cout << "Elemento encontrado\n";
54             return;
55         }
56     }
57     cout << "Elemento no encontrado\n";
58 }
59 template<typename T>
60 void leerDato(T &t) {
61     cin >> t;
62 }

```

```

63 template<
64 void leerDato<string>(string &t) {
65     cin.ignore(numeric_limits<streamsize>::max(), '\n');
66     getline(cin, t);
67 }
68 template<typename T>
69 void menu() {
70     list<T> lista;
71     int op, c = 1;
72     T t;
73
74     while (c != 0) {
75         cout << "\n--- MENU LISTA ---\n";
76         cout << "1. Insertar inicio\n";
77         cout << "2. Insertar final\n";
78         cout << "3. Eliminar inicio\n";
79         cout << "4. Eliminar final\n";
80         cout << "5. Mostrar\n";
81         cout << "Seleccione: ";
82         cin >> op;
83
84         switch (op) {
85             case 1:
86                 cout << "Ingreso dato:\n";
87                 leerDato(t);
88                 lista.push_front(t);
89                 break;
90
91             case 2:
92                 cout << "Ingreso dato:\n";

```

```

93         leerDato(t);
94         lista.push_back(t);
95         break;
96
97     case 3:
98         if (!lista.empty()) {
99             cout << "Eliminado: " << lista.front() << endl;
100             lista.pop_front();
101         } else {
102             cout << "Lista vacia\n";
103         }
104         break;
105
106     case 4:
107         if (!lista.empty()) {
108             cout << "Eliminado: " << lista.back() << endl;
109             lista.pop_back();
110         } else {
111             cout << "Lista vacia\n";
112         }
113         break;
114
115     case 5:
116         mostrar(lista);
117         break;
118
119     default:
120         cout << "Opcion invalida\n";
121     }
122
123     cout << "Continuar? 1 = si / 0 = no: ";
124     cin >> c;
125 }
126 }
127 int main() {
128     int op;
129
130     cout << "--- TIPO DE DATO ---\n";
131     cout << "1. Int\n2. Float\n3. String\n4. Persona\n";
132     cin >> op;
133
134     switch (op) {
135         case 1: menu<int>(); break;
136         case 2: menu<float>(); break;
137         case 3: menu<string>(); break;
138         case 4: menu<persona>(); break;
139         default: cout << "Opcion invalida\n";
140     }
141
142     return 0;
143 }

```

Práctica 10 – Promedio con puntero al arreglo (parcial 1)

Funcionamiento

Se ingresan 5 números utilizando un arreglo, pero accediendo a ellos mediante punteros (acceso indirecto). Con esos valores se calculan: promedio, media, valor máximo, mínimo y suma total. El puntero recorre el arreglo para realizar las operaciones sin usar directamente los índices.

Errores

- No inicializar correctamente el puntero.
- Desbordamiento del arreglo.
- No actualizar correctamente máximo y mínimo.

```
1  #include "Estadisticas.h"
2  #include <iostream>
3
4  using namespace std;
5
6  int main()
7  {
8      Estadisticas e;
9      e.ingresarDatos();
10     cout << "Promedio o media aritmetica: " << e.calcularPromedio() << endl;
11     cout << "Maximo: " << e.obtenerMaximo() << endl;
12     cout << "Minimo: " << e.obtenerMinimo() << endl;
13     cout << "Suma: " << e.calcularSuma() << endl;
14     return 0;
15 }
```

```

1  #ifndef ESTADISTICAS_H
2  #define ESTADISTICAS_H
3
4
5  class Estadisticas
6  {
7  public:
8      Estadisticas();
9      ~Estadisticas();
10
11     void ingresarDatos();
12     int  calcularSuma();
13     float calcularPromedio();
14     int  obtenerMaximo();
15     int  obtenerMinimo();
16
17 protected:
18
19 private:
20     int *numeros;
21 };
22
23 #endif // ESTADISTICAS_H

```

```

1  #include "Estadisticas.h"
2  #include <iostream>
3
4  using namespace std;
5
6  Estadisticas::Estadisticas():numeros(new int[5]) {}
7
8  Estadisticas::~Estadisticas() {}
9
10 void Estadisticas::ingresarDatos()
11 {
12     for(int i = 0; i < 5; i++)
13     {
14         cout << "Ingresa el numero " << i + 1 << ": ";
15         cin >> *(numeros+i);
16     }
17 }
18
19 int Estadisticas::calcularSuma()
20 {
21     int s = 0;
22     for(int i = 0; i < 5; i++)
23     {
24         s += *(numeros + i);
25     }
26     return s;
27 }
28
29 float Estadisticas::calcularPromedio()
30 {
31     return calcularSuma()/5.0;
32 }

```

```

33
34 int Estadisticas::obtenerMaximo()
35 {
36     int max = *numeros;
37     for(int i = 1; i < 5; i++)
38     {
39         if(max < *(numeros + i))
40         {
41             max = *(numeros + i);
42         }
43     }
44     return max;
45 }
46
47 int Estadisticas::obtenerMinimo()
48 {
49     int min = *numeros;
50     for(int i = 1; i < 5; i++)
51     {
52         if(min > *(numeros + i))
53         {
54             min = *(numeros + i);
55         }
56     }
57     return min;
58 }

```

Práctica 13 – Nuevo tipo de dato con puntero al arreglo (parcial 1)

Funcionamiento

Se define un nuevo tipo de dato (por ejemplo, estructura de personas o coches) y se almacena en un arreglo. Luego se accede a los elementos mediante punteros, manipulando la información de forma indirecta (por referencia).

Errores

- Mal uso de operadores.
- Punteros sin inicializar o apuntando a memoria inválida.
- Confusión entre copia de datos y referencia.
- Problemas de memoria si se usa asignación dinámica.

```
1  #include "Persona.h"
2  #include "Auto.h"
3  #include <iostream>
4  #include <ostream>
5  #include <string>
6
7  struct DatosAuto{
8      float precio;
9      int anio;
10 };
11
12 struct DatosPersona {
13     std::string nom;
14     std::string apat;
15     std::string amat;
16     int edad;
17     char genero;
18 };
19
20 using namespace std;
21
22 int main () {
23     int np, na;
24     char r;
25     cout << "Introduce el numero de personas a generar: ";
26     cin >> np;
27     cout << "Introduce el numero de carros a generar: ";
28     cin >> na;
29     cout << "1.\tUsar programacion estructurada" << endl;
30     cout << "2.\tUsar programacion orientada a objetos" << endl;
31     cout << "Ingresa una opcion: ";
32     cin >> r;
```

```

33     if(r == '1'){
34         cout << "====Usando estructuras====" << endl;
35         DatosPersona *pes = new DatosPersona[np];
36         for (int i = 0; i < np; i++) {
37             cout << "Nombre de la persona " << i << ": ";
38             cin >> (pes + i)->nom;
39             cout << "Apellido paterno de la persona " << i << ": ";
40             cin >> (pes + i)->apat;
41             cout << "Apellido materno de la persona " << i << ": ";
42             cin >> (pes + i)->amat;
43             cout << "Genero de la persona " << i << " [M/F]: ";
44             cin >> (pes + i)->genero;
45             cout << "Edad de la persona " << i << ": ";
46             cin >> (pes + i)->edad;
47         }
48         DatosAuto *aes = new DatosAuto[na];
49         for (int i = 0; i < na; i++) {
50             cout << "Año del carro " << i << ": ";
51             cin >> (aes + i)->anio;
52             cout << "Precio del carro " << i << ": ";
53             cin >> (aes + i)->precio;
54         }
55         for (int i = 0; i < np; i++) {
56             cout << "Nombre: " << (pes + i)->nom << endl;
57             cout << "Apellido paterno: " << (pes + i)->apat << endl;
58             cout << "Apellido materno: " << (pes + i)->amat << endl;
59             cout << "Genero: " << (pes + i)->genero << endl;
60             cout << "Edad: " << (pes + i)->edad << endl;
61         }
62
63         for (int i = 0; i < na; i++) {
64             cout << "Año: " << (aes + i)->anio << endl;
65             cout << "Precio: " << (aes + i)->precio << endl;
66         }
67         delete [] pes;
68         delete [] aes;
69     } else if(r == '2') {
70         cout << "====Usando clases====" << endl;
71         Persona *ps = new Persona[np];
72         for (int i = 0; i < np; i++) {
73             string nom, amat, apat;
74             char g;
75             int e;
76             cout << "Nombre de la persona " << i << ": ";
77             cin >> nom;
78             cout << "Apellido paterno de la persona " << i << ": ";
79             cin >> apat;
80             cout << "Apellido materno de la persona " << i << ": ";
81             cin >> amat;
82             cout << "Genero de la persona " << i << " [M/F]: ";
83             cin >> g;
84             cout << "Edad de la persona " << i << ": ";
85             cin >> e;
86             (ps + i)->setNombre(nom);
87             (ps + i)->setApat(apat);
88             (ps + i)->setAmat(amat);
89             (ps + i)->setGenero(g);
90             (ps + i)->setEdad(e);

```

```

91     }
92     Auto *as = new Auto[5];
93     for (int i = 0; i < na; i++) {
94         int a;
95         float p;
96         cout << "Año del carro " << i << ": ";
97         cin >> a;
98         cout << "Precio del carro " << i << ": ";
99         cin >> p;
100         (as + i)->setAnio(a);
101         (as + i)->setPrecio(p);
102     }
103
104     for (int i = 0; i < np; i++) {
105         cout << "Nombre: " << (ps + i)->getNombre() << endl;
106         cout << "Apellido paterno: " << (ps + i)->getApat() << endl;
107         cout << "Apellido materno: " << (ps + i)->getAmat() << endl;
108         cout << "Genero: " << (ps + i)->getGenero() << endl;
109         cout << "Edad: " << (ps + i)->getEdad() << endl;
110     }
111
112     for (int i = 0; i < na; i++) {
113         cout << "Año: " << (as + i)->getAnio() << endl;
114         cout << "Precio: " << (as + i)->getPrecio() << endl;
115     }
116     delete [] ps;
117     delete[] as;
118 }
119
120 return 0;

```



```

1  #ifndef AUTO_H
2  #define AUTO_H
3
4  class Auto{
5  protected:
6      float precio;
7      int anio;
8
9  public:
10     Auto();
11     ~Auto();
12
13     void setPrecio(float p);
14     float getPrecio();
15
16     void setAnio(int a);
17     int getAnio();
18
19 };
20
21 #endif

```

```

1  #include "Auto.h"
2  Auto::Auto(){}
3
4  Auto::~Auto(){}
5
6  void Auto::setPrecio(float p){
7      precio = p;
8  }
9
10 float Auto::getPrecio(){
11     return precio;
12 }
13
14 void Auto::setAnio(int a){
15     anio = a;
16 }
17
18 int Auto::getAnio(){
19     return anio;
20 }

```

```

1  #ifndef PERSONA_H
2  #define PERSONA_H
3
4  #include <string>
5
6  class Persona{
7  protected:
8      std::string nom;
9      std::string apat;
10     std::string amat;
11     char genero;
12     int edad;
13
14 public:
15     Persona();
16     ~Persona();
17
18     void setNombre(std::string n);
19     void setApat(std::string ap);
20     void setAmat(std::string am);
21     void setGenero(char g);
22     void setEdad(int e);
23
24     std::string getNombre();
25     std::string getApat();
26     std::string getAmat();
27     char getGenero();
28     int getEdad();
29 };
30
31 #endif

```

```

1  #include "Persona.h"
2
3  Persona::Persona(){}
4
5  Persona::~Persona(){}
6
7  void Persona::setNombre(std::string n){
8      nom = n;
9  }
10
11 void Persona::setApat(std::string n){
12     apat = n;
13 }
14
15 void Persona::setAmat(std::string n){
16     amat = n;
17 }
18
19 void Persona::setGenero(char g){
20     genero = g;
21 }
22
23 void Persona::setEdad(int e){
24     edad = e;
25 }
26
27 std::string Persona::getNombre(){
28     return nom;
29 }
30
31 std::string Persona::getApat(){
32     return apat;
33 }
34
35 std::string Persona::getAmat(){
36     return amat;
37 }
38
39 char Persona::getGenero(){
40     return genero;
41 }
42
43 int Persona::getEdad(){
44     return edad;
45 }

```